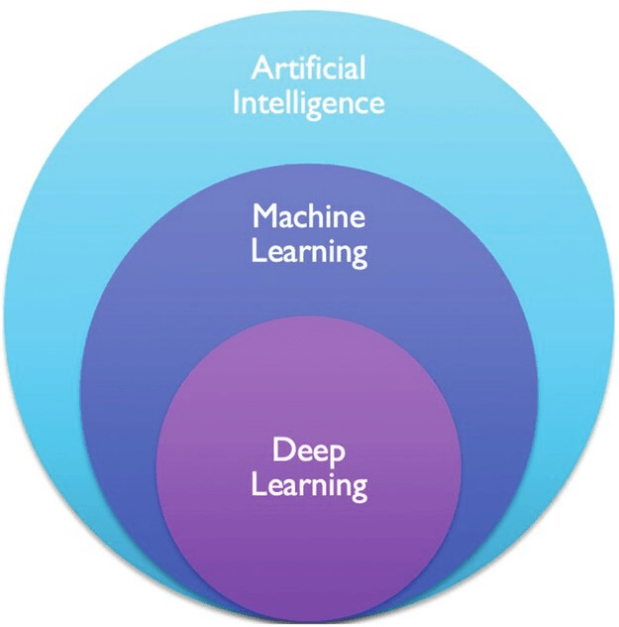
# Class - 01

# Machine Learning

Machine Learning is a subset of AI where computers learn patterns from data and use them to make predictions.



# Machine Learning

Machine Learning is categorized into 3 types:

1. **Supervised Learning**
2. **Unsupervised Learning**
3. **Reinforcement learning**

# Supervised Learning

In supervised learning, the computer is trained using labeled data (input-output pairs)

It learns the relationship between inputs (like features) and outputs (like labels).

After training, it predicts outputs for new, unseen inputs.

# Supervised Learning

Supervised Learning is classified into two problems:

## 1. Regression

Regression algorithms are used to predict continuous (numeric) values.

Ex: Predicting price of house based on some given location.

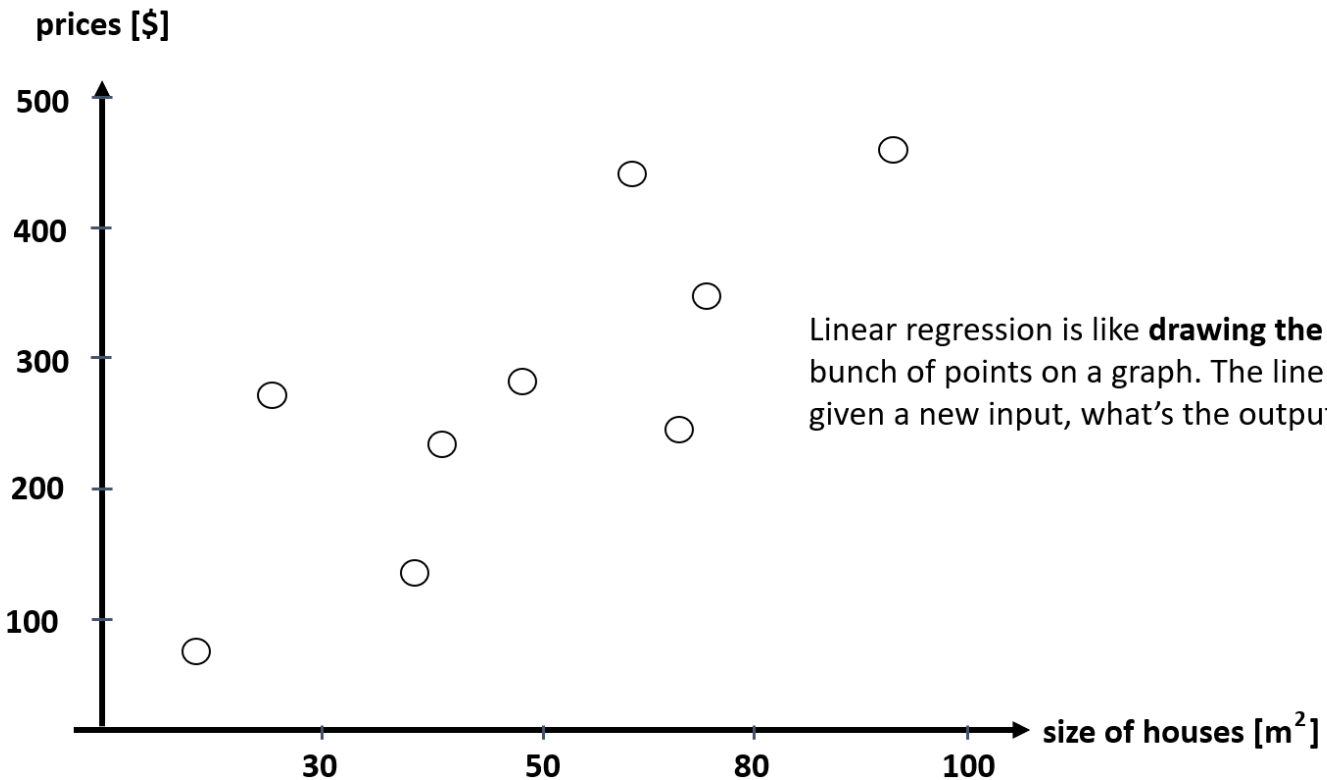
## 2. Classification

Classification algorithms are used to predict categories or labels (like Yes/No, Male/Female, Spam/Not Spam).

# Class - 02

# Linear Regression

# Linear Regression

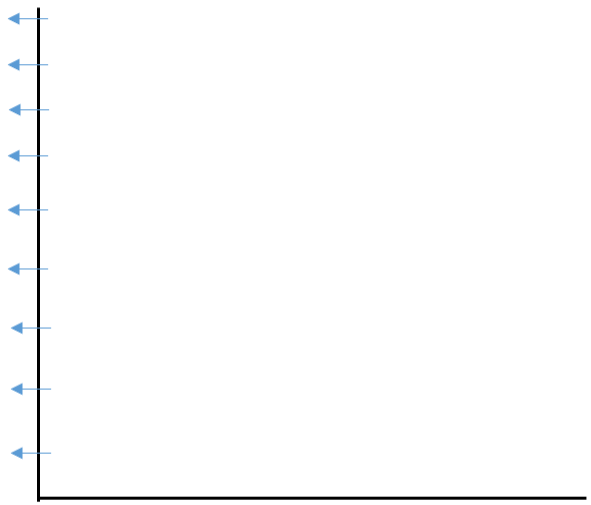



Linear regression is like **drawing the straightest line** through a bunch of points on a graph. The line helps us make predictions: given a new input, what's the output?

# Linear Regression

Lets start with basics lets say you have a plot with X and Y Axis and you want to create a line you decided to first start The line from a point on Y axis.

A line can be created from any point on y axis

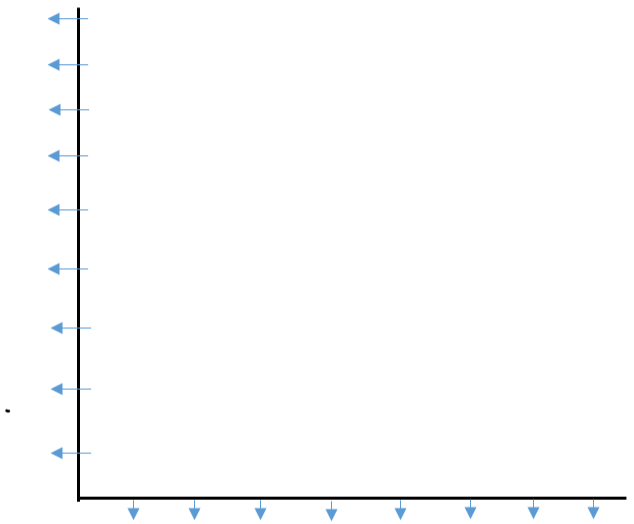


# Linear Regression

Now 2nd step is to set a slope. If you don't know how the Slope is set you just need to understand this.

I was going in straight line when I was starting from a Y point but now as I move forward point by point in x I will increase the value in y as well.

See the demonstration.



# Linear Regression

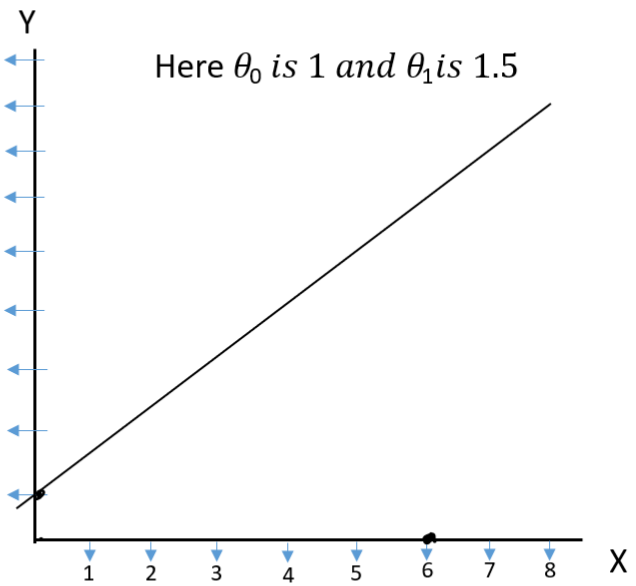
So technically the line depend on the intercept from Where I am starting my line in Y and the slope we have Set.

So for the intercept we will call it $\theta_0$ and for the slope We call it $\theta_1$

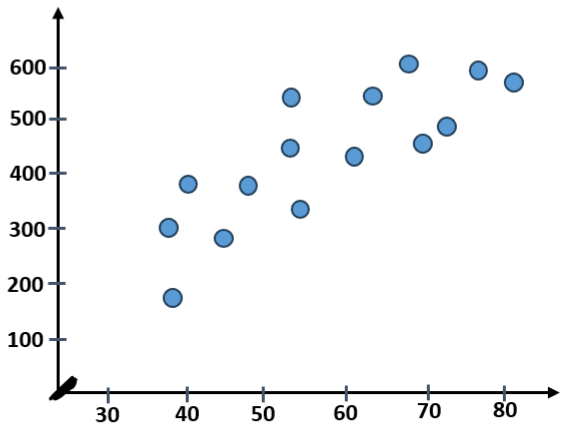
Now once you have created the line now its time to Predict, for prediction we will use a very simple formula And lets see that in action.

$Y = \theta_0 + \theta_1 X$ ➔ formula

Lets see what will be the value of Y if X is 6.



# Linear Regression




But all this is to make line but there can be many lines
Not 1 not 2 they can be ∞

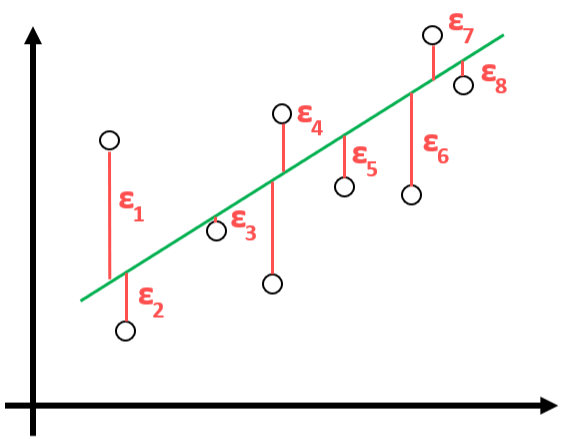
Then which line to choose to choose the best line we will
Use errors.

Now in supervised learning we already have the Input(independent) as well as output (dependant) variable so in our graph we will make a scatter plot.

And on this plot Using $\theta_0$ and $\theta_1$ value we can make any line as Demonstrated.

Now for each line we will calculate the Error Lets see how.

# Linear Regression




Lets just say we have created a line between the scattered Points now we have to calculated the error of that line .

To do so we just have to calculate the distance between the Actual available points and predicted points according to the Line. We will use Cost function for that.

Just calculate all the distance square them up and add all of Them.

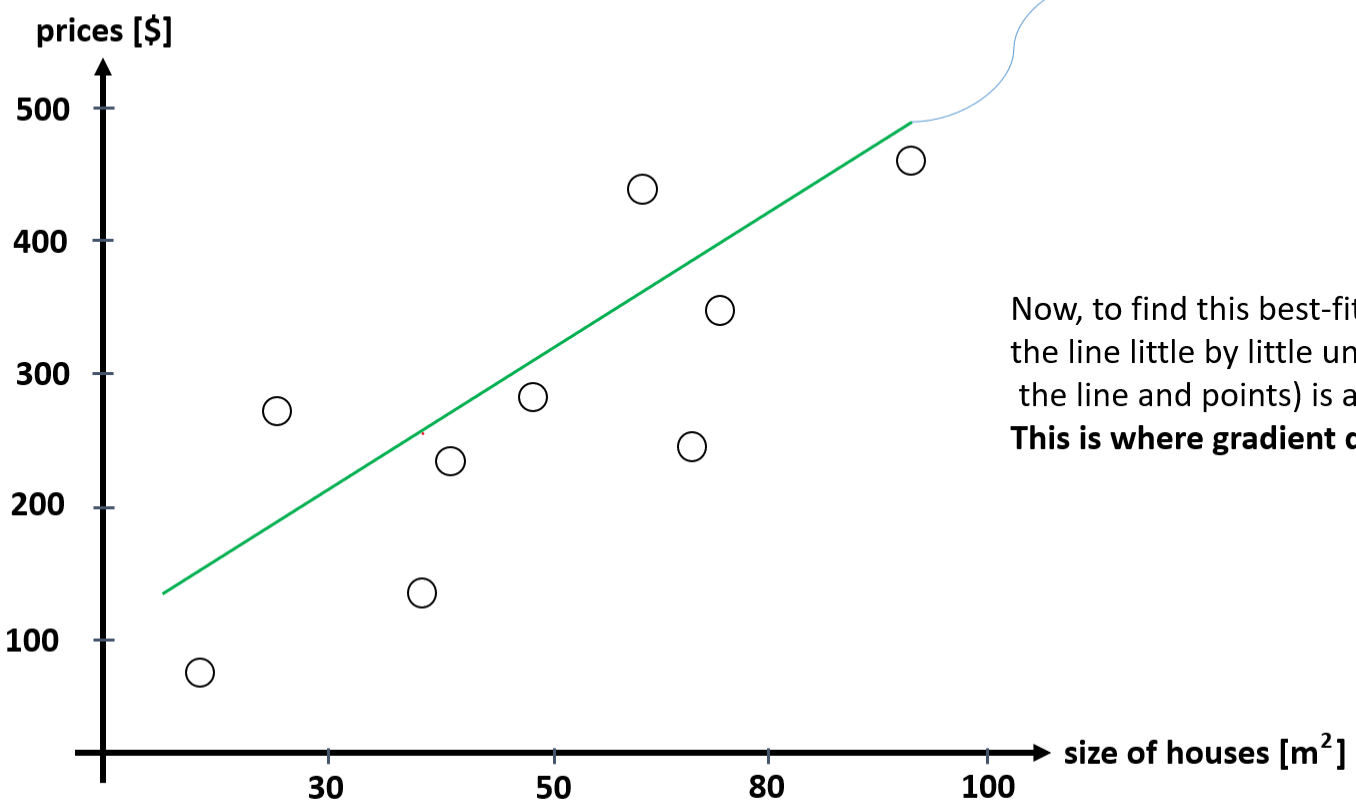
Small Cost ➔ If the value is Low that means the error is low and our predictions will be good.

High Cost ➔ if the value is High that means the error is High and our prediction will be bad.

So formula is simple – $(\bar{y}i - yi)^2$

The term is known as mean squared error – $J(\theta_0, \theta_1) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$

# Linear Regression




This line is known as the best fit line

Now, to find this best-fit line, we need to adjust the line little by little until the error (the gap between the line and points) is as small as possible.

**This is where gradient descent comes in!**

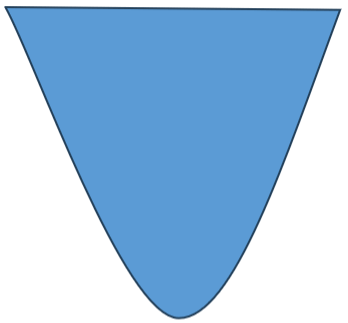
# Linear Regression

So at last our task is very simple we have to reduce the error and make our prediction more perfect
For that we will use OPTIMIZATION.

And for optimizing the values and choosing the best line out of tens and thousands of line we will use
The OG the Goat Gradient Descent. 💕 ❤️ ❤️ ❤️ ❤️

# Gradient Descent

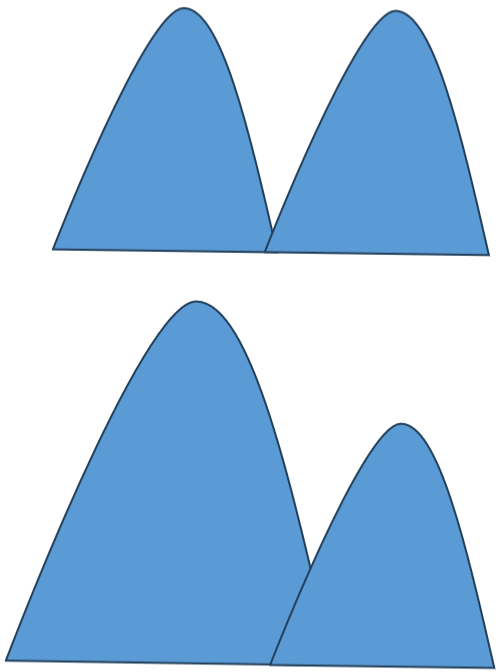
**Gradient** is just a fancy word for slope or steepness of the hill. **Descent** means going down. So, gradient descent means **going down a hill by taking small steps in the direction of the steepest slope**.



# Gradient Descent

Imagine you're standing on top of a big hill. You want to reach the bottom, but it’s foggy, and you can't see far. To avoid falling, you take small steps downwards. Each time you take a step, you check if you're going lower.

The goal is to **find the lowest point** (the bottom of the hill), but since you can't see the whole hill, you use small steps to slowly get there. The steeper the hill, the bigger your steps can be. On flatter parts, you take smaller steps to avoid tripping or overshooting.



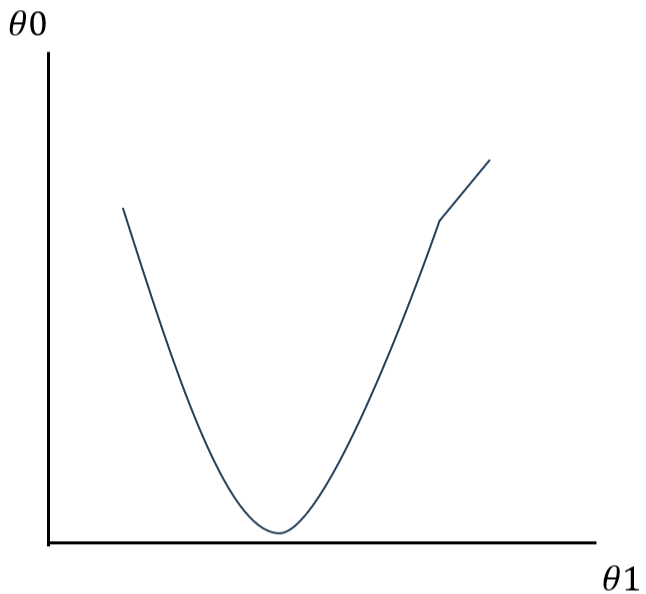
# Gradient Descent

Now lets say you have created a simple line
on a plot with some specific theta 0 and theta 1
Values now we transfer these values to be tested
In gradient descent in which we will take steps down

The steps will be the learning rate –
And Fuck yes the learning rate will be
our significance value $\alpha$

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i) x_i$$

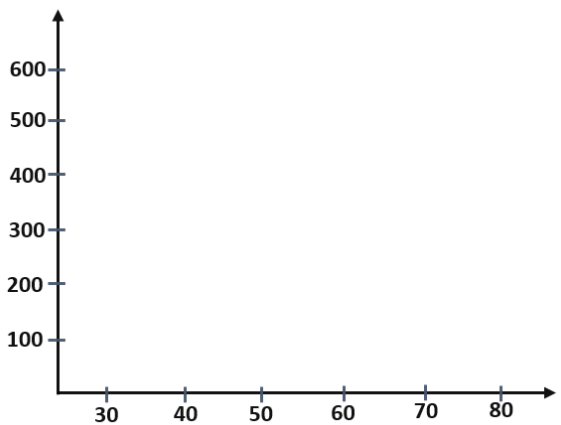


# Linear Regression – Testing

| Price | Sq ft |
|---|---|
| 12k | 128 |
| 13k | 130 |
| 10k | 100 |
| 80k | 600 |
| 40k | 320 |
| 20k | 159 |
| ... | ... |

N = 10 , p = 1

Now first lets understand what is degree of freedom and How the R square and Adjusted R square works.



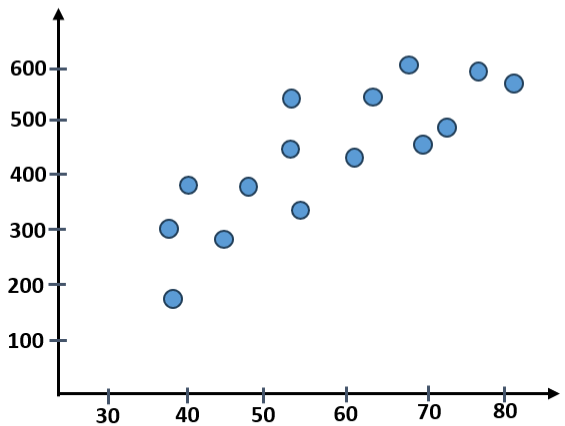

Degree of freedom means, freedom to find error.

# Linear Regression – Testing

Think of **R-squared** as the score for your model that tells you how well it's doing. It shows how much of the actual result your model can explain.

R square value comes between 0 and 1 and it represents the Percentage how well your model is working.

Formula ➔ $$\frac{\text{Var(mean)} - \text{var(best fit line)}}{\text{Var(mean)}}$$



# Linear Regression – Testing

Now, sometimes R-squared can get a little tricky. If you add too many things (features) to your model, R-squared might say “Yay, I’m getting better!” even if the new features don’t actually help.
This is where **Adjusted R-squared** comes in—it’s smarter! It checks if the extra stuff you’re adding is actually helping. If not, it "punishes" the model by lowering the score a bit.

$$\text{Adjusted } R^2 = 1 - \left( \frac{(1 - R^2)(n - 1)}{n - p - 1} \right)$$

n – number of data points.
P – number of features.

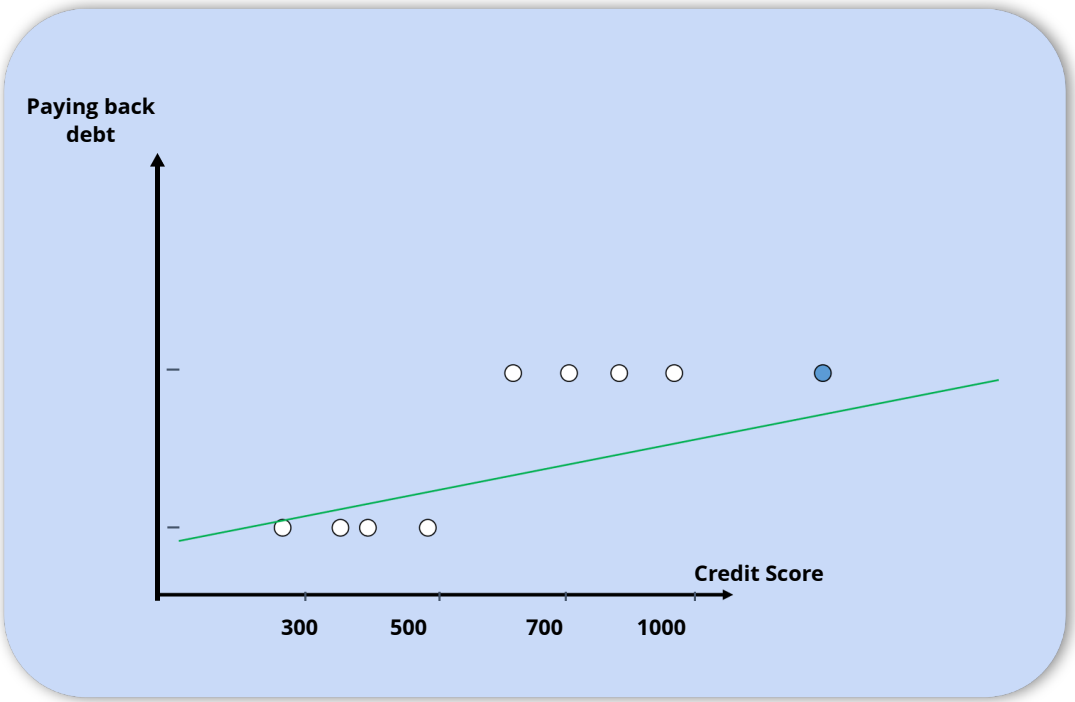
# Class - 03

# Logistic Regression

**Linear Regression Problem:**

The issue with Linear Regression is that if a new data point (called an outlier) is added, the best-fit line can change a lot.

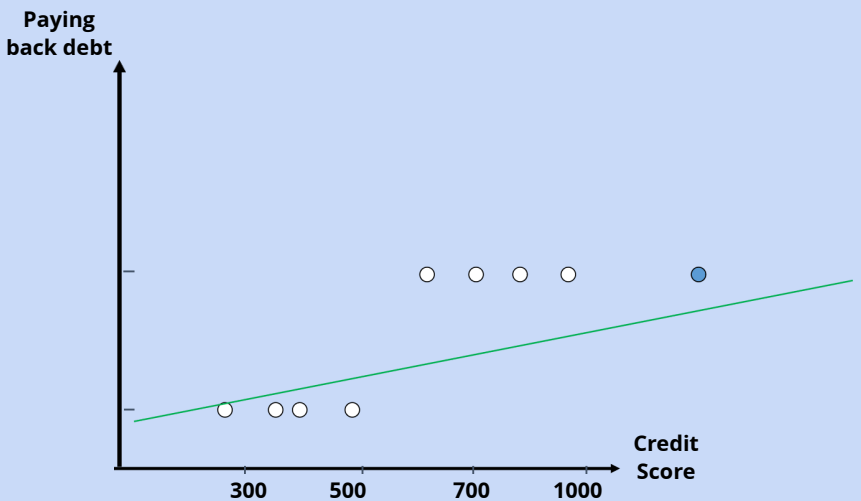
This makes the model unstable and less accurate for predictions.



In Logistic Regression, probabilities are used to classify data into categories.

If the probability is **greater than 50%**, the model predicts **“Yes”** otherwise, it predicts **“No”**.

| Income($) | Credit Score | Loan |
|---|---|---|
| 40,000 | 750 | Yes |
| 25,000 | 600 | No |
| 50,000 | 800 | Yes |
| 30,000 | 580 | No |



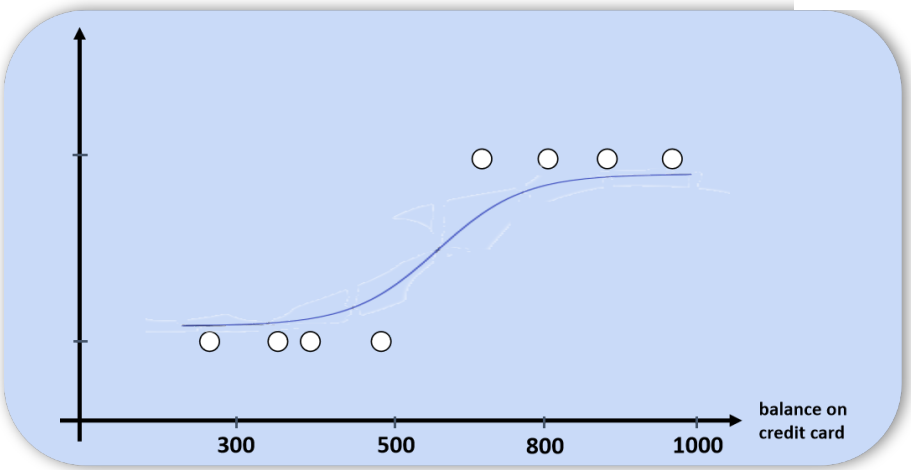
So now you all might think how is this S-Curved line is formed?

Here we are using a sigmoid function

$$f(z) = \frac{1}{1 + e^{-z}}$$

**what this sigmoid function will helpful for ?**

This function will squash the line from Linear Regression into a range between 0 and 1, making it suitable for classification tasks by transforming the outputs into probabilities.



In Linear Regression we measure how far away predictions are from actual values by using Mean Squared Error, or MSE.

In Logistic Regression, we cannot use MSE since we predict a probability between 0 and 1, and not a continuous number. Thus, we will use Log Loss.

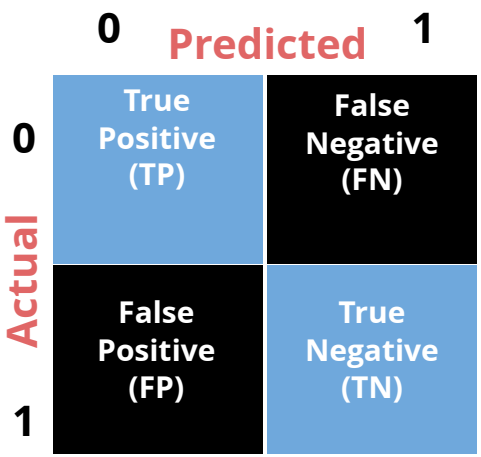
$$\textbf{Log Loss} = -(y.\log(p) + (1-y).\log(1-p))$$

Here y is the actual value like 0 or 1 and here p is the predicted probability

# Logistic Regression

**Accuracy Score**: Measures the percentage of correct predictions out of all predictions.

$$\frac{\text{True Positive} + \text{True Negative}}{\text{Total Predictions}}$$




**Precision:** Measures how many of the predicted positives are actually correct.

$$\frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

**F1 Score:** The harmonic mean of precision and recall, balancing both metrics.

$$2 \times \frac{\text{Precision} + \text{Recall}}{\text{Precision} \times \text{Recall}}$$

# Class - 04

# K - Nearest Neighbor

# K Nearest Neighbor

K-Nearest Neighbors (KNN) is a simple algorithm used for classification and regression.

It makes predictions by looking at the K nearest data points using distance, like Euclidean distance.

KNN doesn’t build a model, it just stores the data, which makes it easy to understand but slow for large datasets.

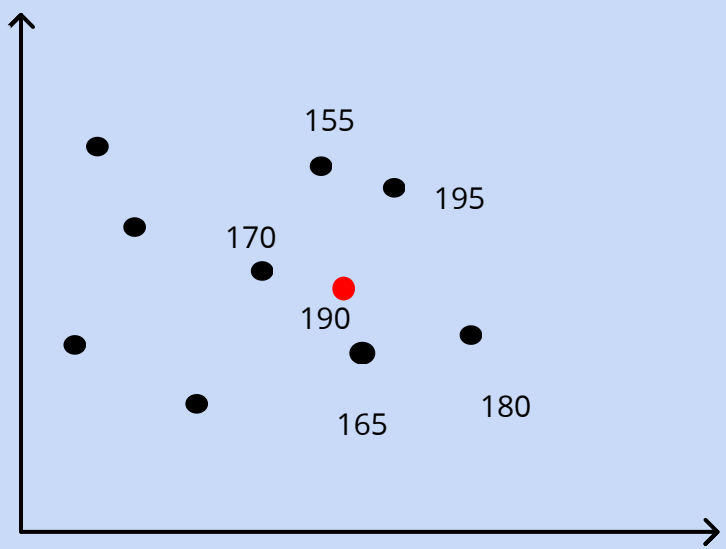
Now we can use KNN for regression problem and for classification problem.

# K Nearest Neighbor Regressor

Now for some time let's just assume that the K-value we are having is 5 and the new data point we are having is 190.

What does happen at this time that 5 nearest value will be calculated near the new data points.

| Height(cm) | Weight(Kg) | Class |
|---|---|---|
| 170 | 55 | fit |
| 165 | 68 | fit |
| 180 | 75 | unfit |
| 195 | 79 | fit |
| 155 | 50 | unfit |



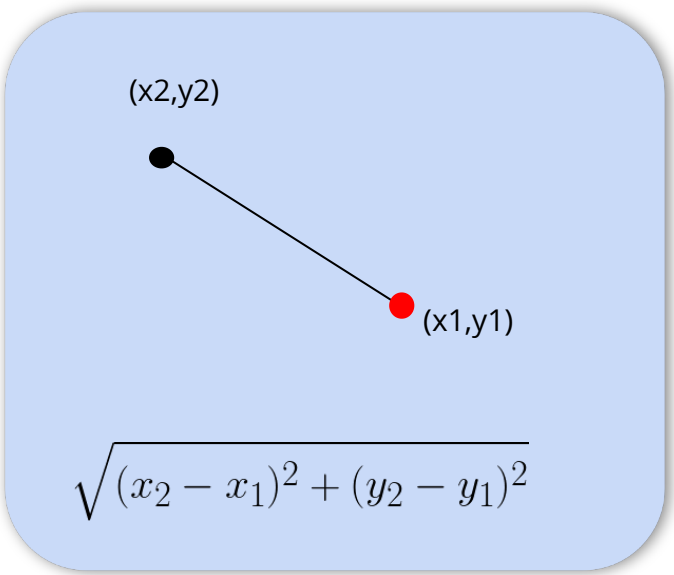
# K Nearest Neighbor Regressor

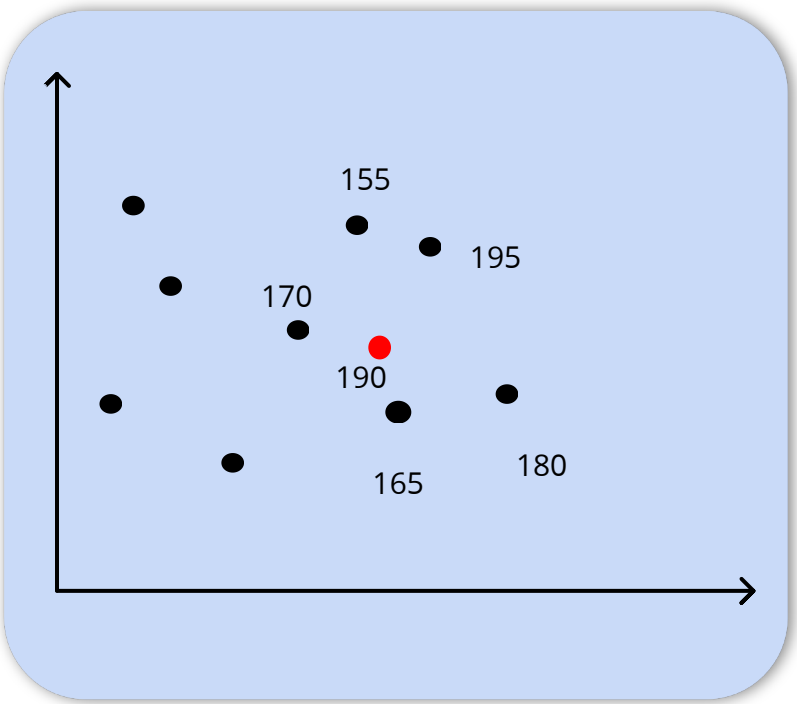
Now you must be thinking how do we calculate the Euclidian Distance

we calculate Euclidian distance for two points A and B with coordinates and after calculating, we select the 5 nearest points.

The Euclidian distance between d(A, B) is given by:

$$d(A,B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



# K Nearest Neighbor Regressor

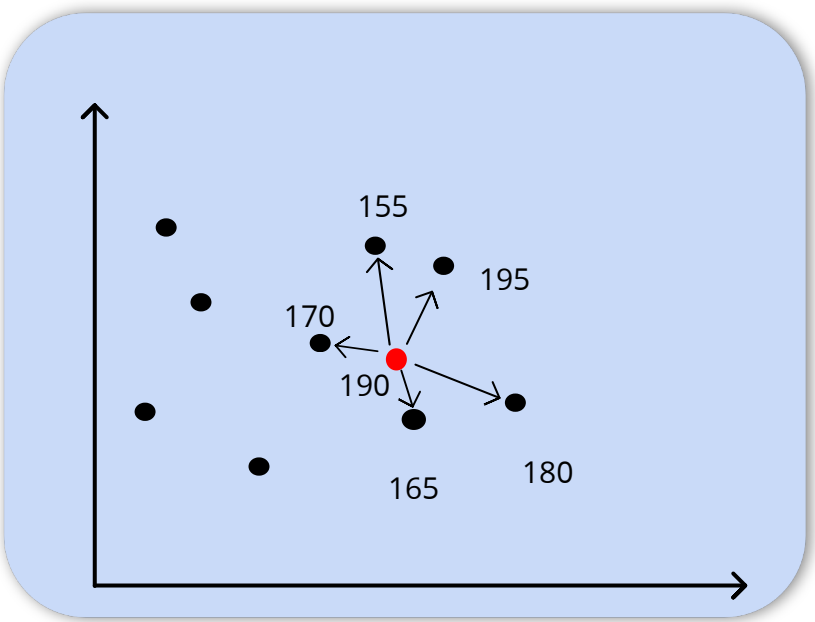
For regression we will take average of these 5 selected points near the new data point 190

so the average of these 5 selected values will be:

$$y = \frac{170 + 155 + 195 + 180 + 165}{5} = 173$$

so the predicted value for **x = 190** will be **y = 173**

| Height(cm) | Weight(Kg) | Class |
|---|---|---|
| 170 | 55 | fit |
| 165 | 68 | fit |
| 180 | 75 | unfit |
| 195 | 79 | fit |
| 155 | 50 | unfit |



# K Nearest Neighbor Classifier

The K-Nearest Neighbors (KNN) Classifier is a supervised learning algorithm used to classify data.

Now the steps will be going to same as we are using for KNN regressor.

We will again going to select some K values  and then we will select some K nearest points.

But one thing will be changed that we are not going to take average this time instead we are going to make votings.

Let's see how we do it.

# K Nearest Neighbor Classifier

Let's say we choose k-value = 3

After calculating distances, you find the 3 closest fruits.

Fruit 1 (Apple): Distance = 0.2

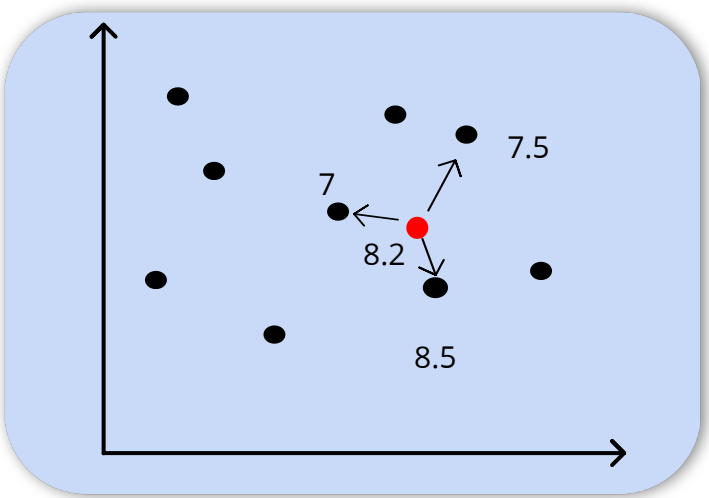
Fruit 4 (Orange): Distance = 0.3

Fruit 2 (Apple): Distance = 0.4

Among the 3 nearest neighbors, 2 are Apple and 1 is Orange.

Since Apple appears more frequently, the new fruit having

Size 8.2 and weight 175g will be classified as Apple.



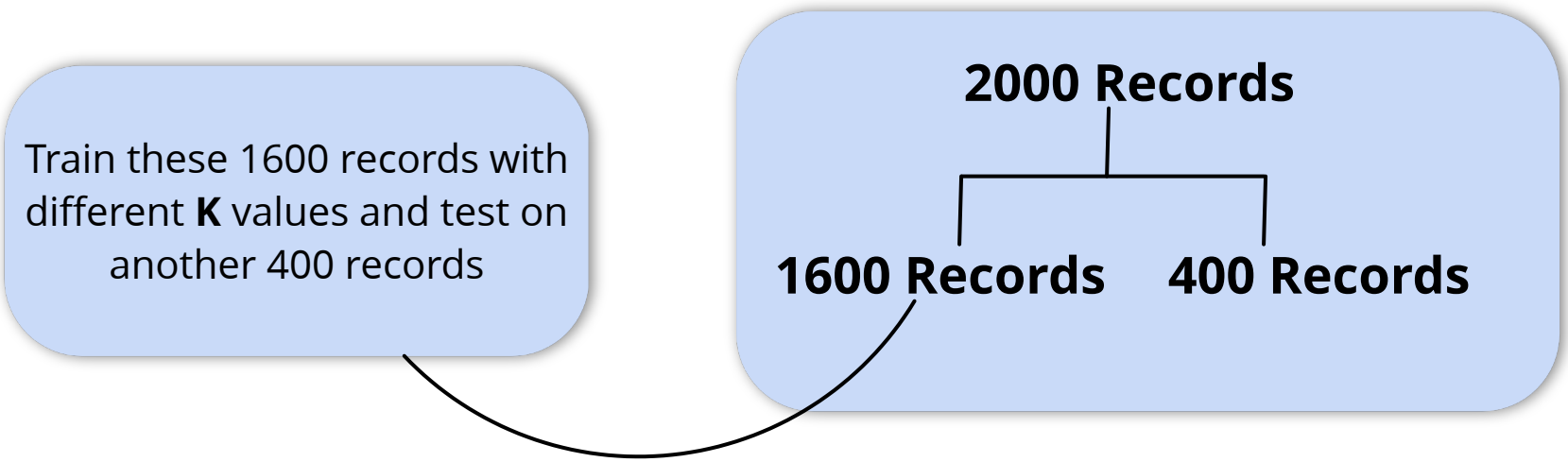

| Fruit Id | Size | Weight(gram) | Class (Fruit Type) |
|---|---|---|---|
| 1 | 7 | 150 | Apple(A) |
| 2 | 7.5 | 160 | Apple(A) |
| 3 | 8 | 170 | Apple(A) |
| 4 | 8.5 | 180 | Orange(O) |

# K Nearest Neighbor

**How do we select the K-values?**

To find the best K, we use **Cross-Validation**.

- Split the dataset into training and validation sets.
- Test different values of K.
- Choose the K that gives the **highest accuracy** on validation data.



# Class - 05

# Naive Bayes

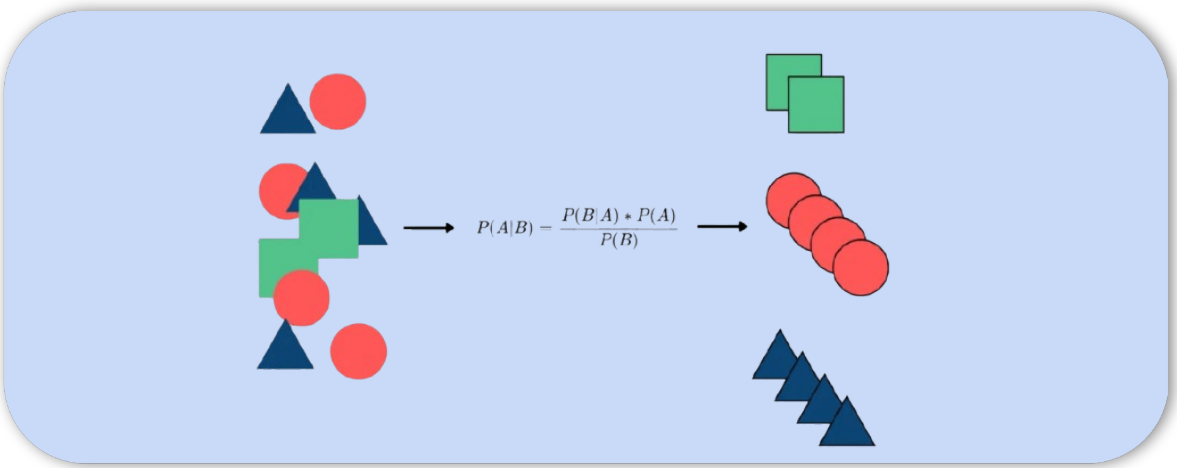
# Naive Bayes

Naive Bayes is a simple yet powerful probabilistic algorithm used for classification tasks which is based on Bayes' Theorem.

The primarily use of Naive Bayes is for  classification tasks, not regression.

This is because Naive Bayes is a probabilistic classifier based on Bayes' Theorem, and it predicts discrete class labels rather than continuous values.

Bayes' Theorem provides a way to calculate the probability of a class given a set of features:



# Naive Bayes Classifier

Naive Bayes is a probabilistic algorithm used for classification tasks.

It is based on Bayes' Theorem and assumes that all features are conditionally independent given the class label.

Despite its simplicity, it is effective for a wide range of classification problems, especially text classification.

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

This is Bayes' Formula, which is the foundation of the Naive Bayes algorithm and is used to make predictions.

# Naive Bayes Classifier

Now here we will calculate Bayes theorem for this new white data point .

For this we need to calculate some parameters and what are they?

The formula we know for Naive Bayes.

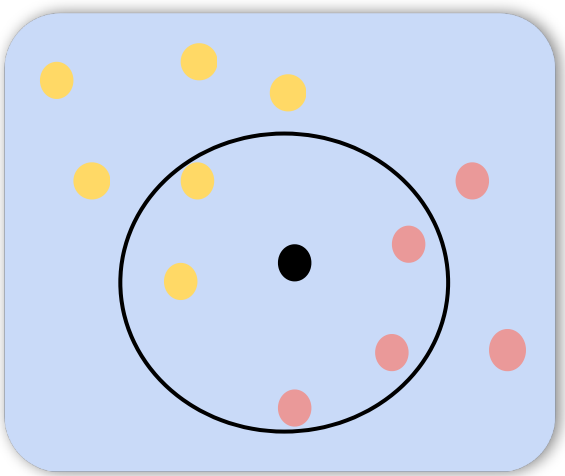
Now, we have to calculate all the parameters require for formula

P(Yellow) = 6/11

P(Red | Yellow) = 2/6

P(Blue) = 5/11

P(Red | Blue) = 3/5



# Naive Bayes Classifier

Let's calculate the probability of both events (Red and Yellow) happening together and same for (Red and Blue).

**P(Red n Yellow) = P(Yellow) x P(Red | Yellow)**

**P(Red n Blue) = P(Blue) x p(Red | Blue)**

To find the total probability of Red, we add the chances of Red with Yellow and Red with Blue:

**P(Red) = P(Red n Yellow) + P(Red n Blue) = 12/66 + 15/66 = 27/66**

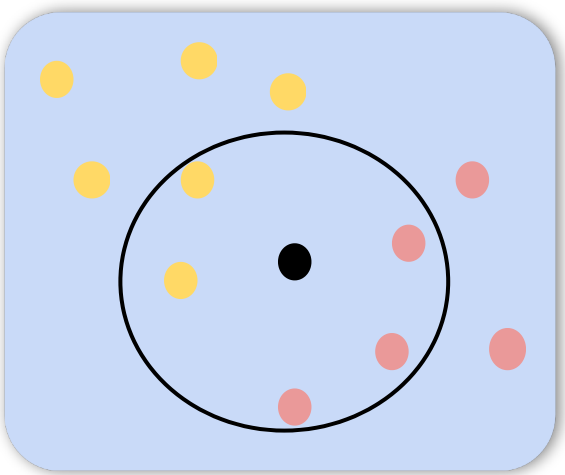
# Naive Bayes Classifier

Calculating the probability of Yellow and Blue occurring given that Red has already occurred.

**P(Yellow | Red) = P(Red n Yellow)/P(Red) = 0.44**

**P(Blue | Red) = P(Red n Blue) / P(Red) = 0.56**

Here Probability of P(Blue | Red) > P(Yellow | Red) so, the new Red ball belongs to **class of Blue balls**



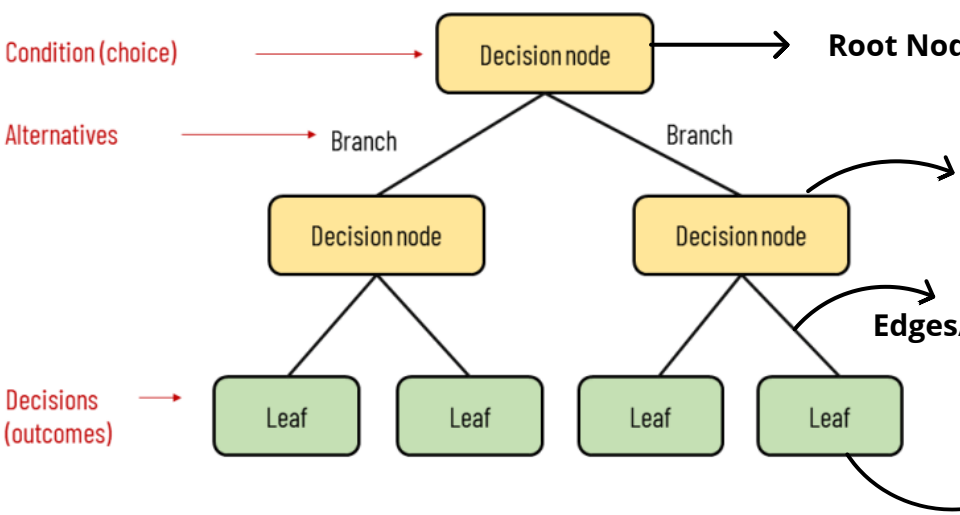
# Class - 06

# Decision Tree

# Decision Tree

A Decision Tree is a supervised machine learning algorithm that is used for classification and regression tasks.

It works by splitting the data into subsets based on feature values, ultimately making predictions through decision nodes (branches) and leaf nodes.




**Root Node**: The first node that represents the whole dataset.

**Decision Node**: A node where data is split using a condition.

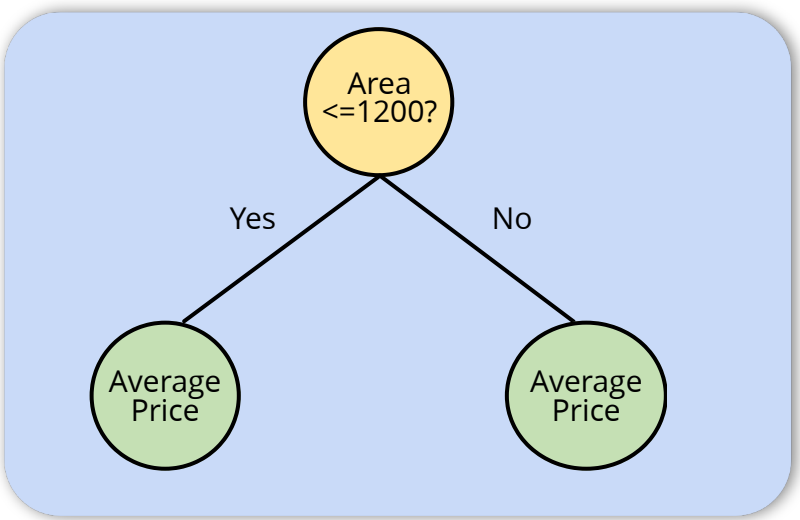
**Edges/Branches**: Lines that connect nodes, showing decision outcomes.

**Leaf Node**: The end node that gives the final result (class/ value).

# Decision Tree Regression

A **Decision Tree Regression** is a supervised machine learning algorithm used for predicting continuous values.

It works by partitioning the data into subsets based on feature values, making predictions at the leaf nodes, and minimizing prediction error.

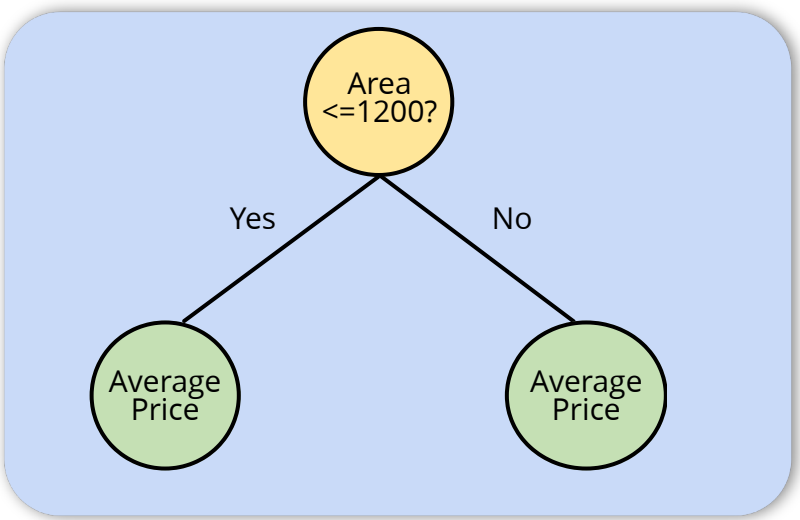


# Decision Tree Regression

A **Decision Tree Regression** is a supervised machine learning algorithm used for predicting continuous values.

It works by partitioning the data into subsets based on feature values, making predictions at the leaf nodes, and minimizing prediction error.

Let's see this with some sample dataset
We will use Area (sqft) as the feature for splitting. Let’s say we first split at 1200 sqft. means we are splitting our decision tree on the basis of Area(sqft)




| House Id | Area(sqft) | Price($1000) |
|---|---|---|
| 1 | 800 | 200 |
| 2 | 1000 | 250 |
| 3 | 1200 | 270 |
| 4 | 1500 | 300 |
| 5 | 1800 | 350 |
| 6 | 2000 | 380 |

# Decision Tree Regression

Let's calculate Average Price for House which are less that 1200 sqft and the house which are not less then 1200sqft

**Left Node (Area ≤ 1200 Sqft):**

Houses: House 1 (800 sqft), House 2 (1000sqft), House 3 (1200 sqft)

Prices: 200, 250, 270

Average Price = $\frac{200+250+270}{3}$

Average Price = 240

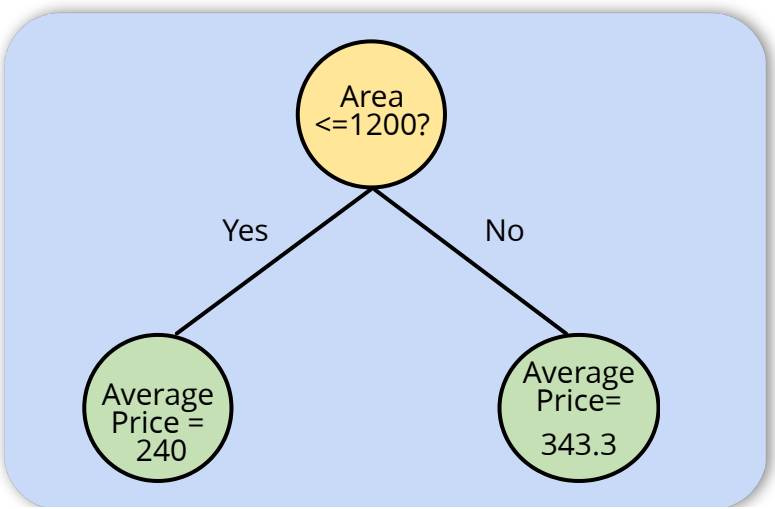
**Righ Node(Area > 1200 Sqft)**

Houses: House 4 (1500 sqft), House 5 (1800 sqft), House 6(2000 sqft)

Prices: 300, 350,380

Average Price = $\frac{300 + 350 + 380}{3}$

Average Price = 343.3

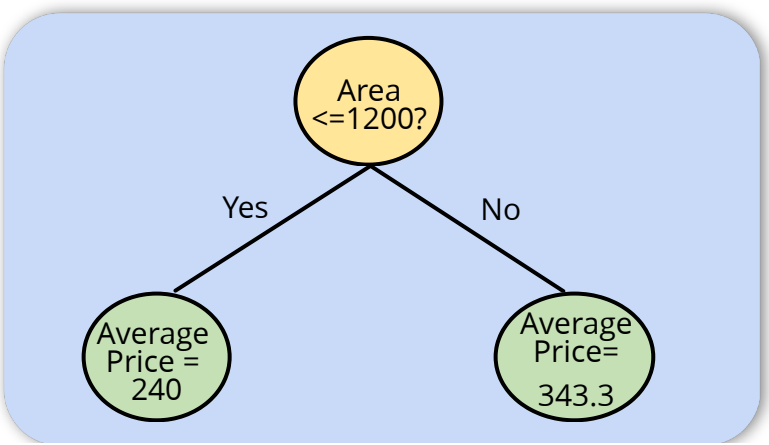



| House Id | Area(sqft) | Price($1000) |
|---|---|---|
| 1 | 800 | 200 |
| 2 | 1000 | 250 |
| 3 | 1200 | 270 |
| 4 | 1500 | 300 |
| 5 | 1800 | 350 |
| 6 | 2000 | 380 |

# Decision Tree Regression

Given a new data point with Area = 1300 sqft, we follow these steps to predict its price:

- Root Node Check: The first condition in the root node checks if the area is ≤ 1200 sqft or > 1200 sqft.
- Since 1300 > 1200, it will satisfy the No condition and move to the right node.
- Right Node (Area > 1200 sqft): For houses with an area greater than 1200 sqft, the predicted price is the average price for that subset of houses.
- Thus, the predicted price for a house with Area = 1300 sqft will be 343.3, which is the average price of houses with an area greater than 1200 sqft.
- This is how the decision tree applies the conditions and uses the average price of the relevant group to make the prediction.



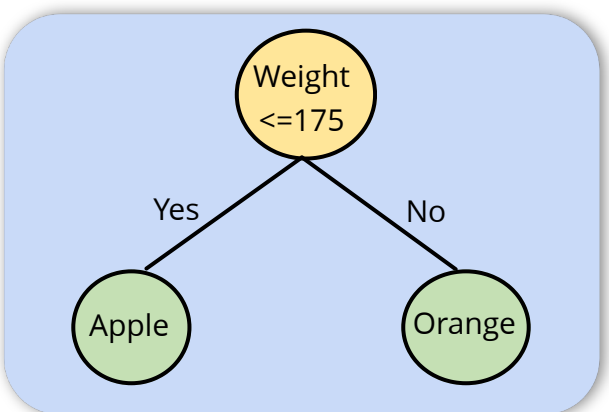
# Decision Tree Classification

A Decision Tree Classification is a supervised learning algorithm used for classification tasks, where the goal is to predict a categorical label.

It splits the data into subsets based on the feature values and ultimately classifies the data based on the majority class in the leaf nodes.

Let's get the steps for Classification:

1. Select the Best Feature: The first step is to find the best feature to split the dataset.
2. Split the Data: Based on the best feature, the data is divided into subsets (branches).
3. Leaf Nodes: When the tree reaches a leaf node, it assigns the most common class label for that subset of data.



# Decision Tree Classification

Let's consider a sample dataset for Decision Tree Classification, where we classify fruits based on their Fruit ID, Size, and Weight. The goal is to determine whether a given fruit is an Apple or an Orange.

We will use Weight as the primary splitting criterion:

- If the weight is greater than 175g, the fruit is classified as Orange.
- If the weight is less than or equal to 175g, the fruit is classified as Apple.

**Decision Rule:**

- **If** `Weight > 175g` → **Orange**
- **If** `Weight ≤ 175g` → **Apple**

| Fruit Id | Size | Weight(g) | Class (Fruit Type) |
|---|---|---|---|
| 1 | 7 | 150 | Apple(A) |
| 2 | 7.5 | 160 | Apple(A) |
| 3 | 8 | 170 | Apple(A) |
| 4 | 8.5 | 180 | Orange(O) |
| 5 | 9 | 190 | Orange(O) |

# Decision Tree

When constructing a Decision Tree, we need to decide which feature to split on at each step.

To determine the best split, we use impurity measures like Gini Index and Information Gain.

## 1. Gini Index

The Gini Index measures the impurity of a dataset. It calculates how often a randomly chosen element would be incorrectly classified if it were randomly labeled according to the class distribution.

$$Gini\ Impurity = 1 - \sum_{i=1}^{K} p_i^2$$

$p_i$ is the probability of a class in a given node.

## Interpretation:

- **Gini = 0** → Pure node (all elements belong to one class).
- **Higher Gini** → More impurity, meaning more mixed classes.
- **Goal:** Minimize Gini to get the best split.

# Decision Tree

## 2. Information Gain

Information Gain is used in Decision Trees to select the best feature for splitting the data at each node.

It helps in determining which feature provides the most useful information for classification or regression.

Information Gain tells us how much uncertainty (entropy) is reduced after a split.

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

$p_i$ is the probability of a each class.

**Information Gain:**

$$IG = \text{Entropy}_{\text{Parent}} - \sum \left( \frac{\text{Nparent}}{\text{Nchild}} \text{ x Entropy Child} \right)$$

# Class - 07

# Prunning and Bagging

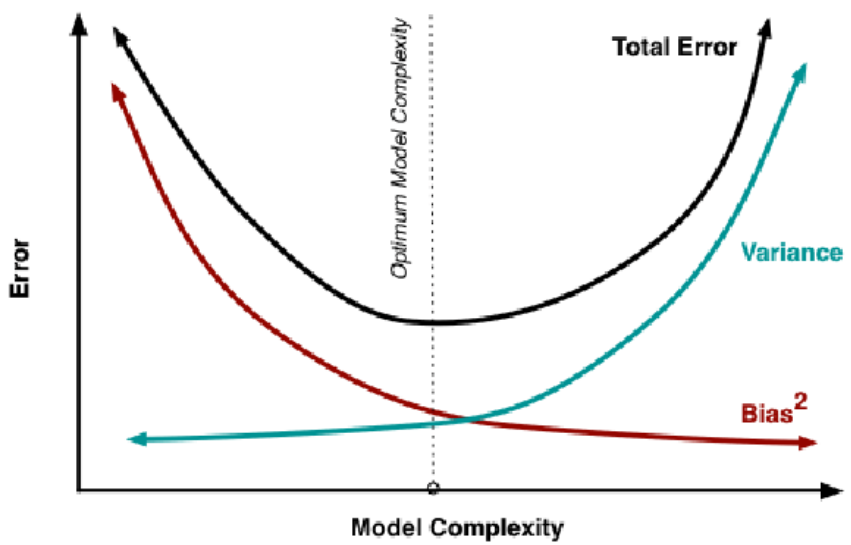
# Prunning and Bagging

When we create a model, our goal is to ensure that it performs well on training data while also generalizing effectively to unseen test data.

However, in the case of Decision Trees, achieving both simultaneously is quite challenging.

This is where the bias-variance trade-off comes into play.

To strike the right balance between bias and variance, we use techniques like pruning and bagging. But before diving into these techniques, let's first understand what bias and variance mean.



# Prunning and Bagging

## Bias and Variance

- Bias → Error due to incorrect assumptions in the learning algorithm.
- High Bias → The model fails to capture the underlying patterns in the data, leading to underfitting.
- Variance → The sensitivity of the model to fluctuations in the training data.
- High Variance → The model memorizes noise in the data, performing well on training but poorly on unseen data (overfitting).

## Bias-Variance Trade-off

- Low Bias → High Variance (Overfitting)
- Variance → High Bias (Underfitting)

The goal is to balance bias and variance to improve model generalization, using techniques like pruning and bagging in decision trees.

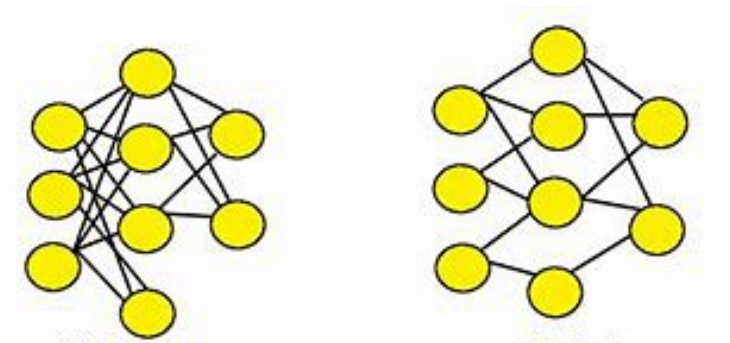
# Prunning

Pruning is a technique in machine learning that involves diminishing the size of a prepared model by eliminating some of its parameters.

The objective of pruning is to make a smaller, faster, and more effective model while maintaining its accuracy.

## Types of Pruning :

- **Pre-Pruning:** Stops growth early based on depth, sample size, or gain threshold.
- **Post-Pruning:** Grows a full tree, then removes weak branches.
- **Cost Complexity Pruning (CCP):** Penalizes large trees to reduce overfitting.



Before Prunning

After Prunning

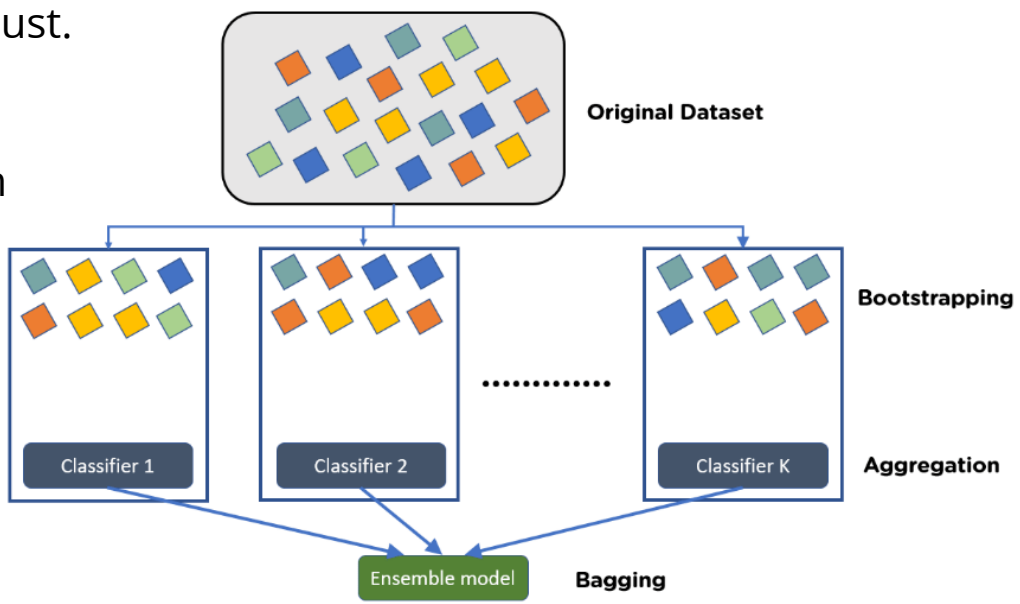
# Bagging

Bagging is an ensemble learning technique that improves model stability and accuracy by **reducing variance**.

It combines multiple models trained on different random subsets of data to make robust predictions.

Bagging is particularly effective for high-variance models like decision trees, preventing overfitting and making predictions more robust.

A well-known example of bagging is Random Forest, which builds multiple decision trees and aggregates their outputs for better performance.



# Bagging

we should take repeated samples from the single data set + construct trees + average

all the predictions in the end all the trees are fully grown unpruned decision trees

**THIS IS CALLED BAGGING**

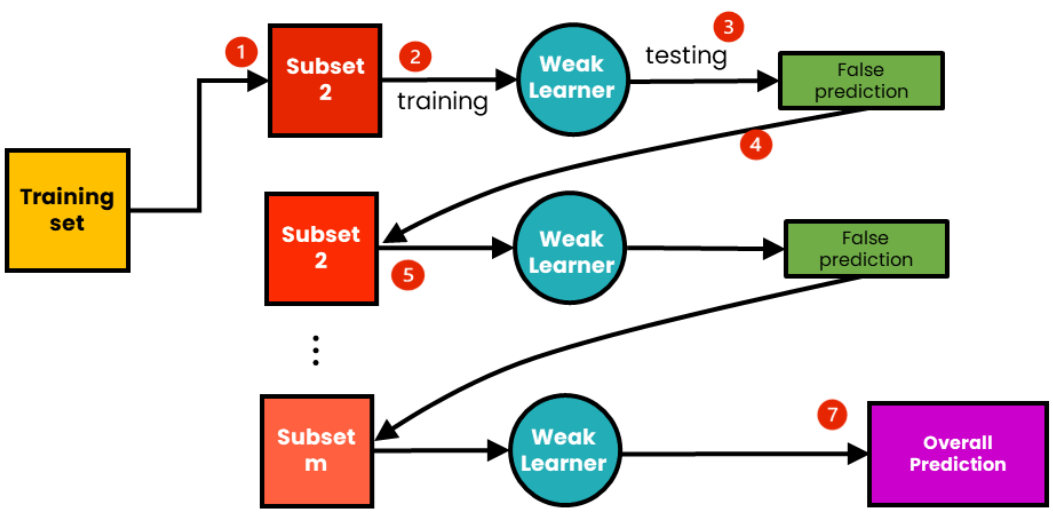
Regression problem : we take the average.

Classification problem : we take the majority vote.

# Boosting

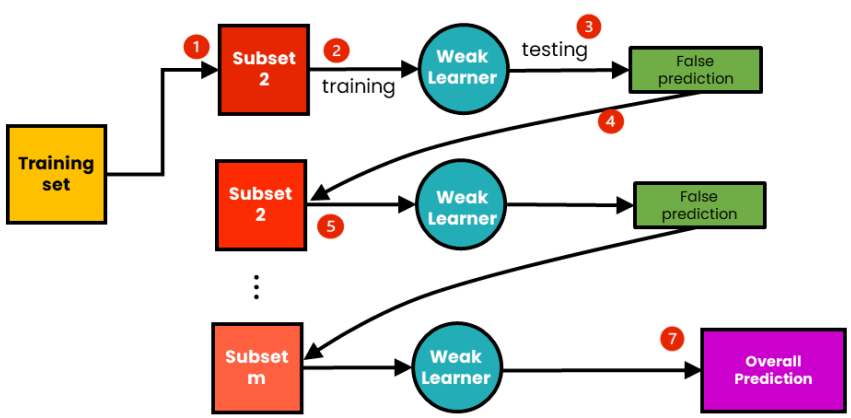
Boosting is a **sequential learning algorithm** where decision trees are grown **one after another**, and each new tree is trained to correct the errors made by the previous trees.

The key difference from **bagging** is that the trees in boosting are **not independent**, as each tree relies on the information and errors of the preceding trees.



# Boosting

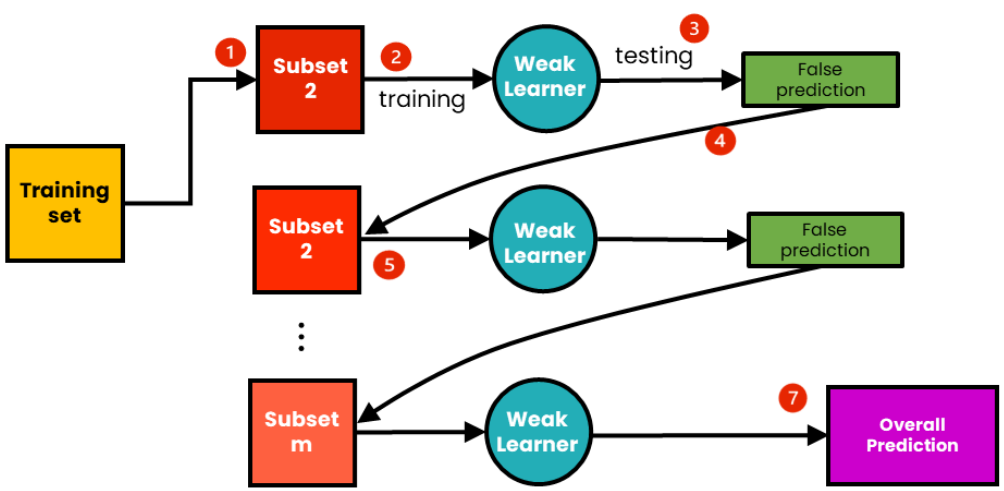
- **Weak Learner:** A simple model with limited accuracy, like a shallow tree.
- **Strong Learner:** A powerful model that combines multiple weak learners for better accuracy.
- **Creates a strong learner**: Combines weak learners (shallow trees) to improve performance.
- **Focuses on misclassified data**: Gives more importance to misclassified points, improving accuracy on harder cases.
- **Reduces bias and variance**: Improves predictions (**reduces bias**) and addresses difficult cases (reduces variance).
- **Sensitive to noise**: Can overfit noisy data.



# Boosting

**Steps in Boosting**:

1. **Train the first tree**: Learn from the entire dataset.
2. **Assign weights to misclassified instances**: These weights increase the importance of the errors.
3. **Train subsequent trees**: Focus on the misclassified points, updating the weights and improving accuracy.
4. **Combine predictions**: The final model prediction is the weighted sum of predictions from all the trees.



# Boosting

**Some Main Points about Boosting:**

- **Sequential process**: Trees depend on one another, unlike in bagging.
- **Improved accuracy**: By focusing on misclassified data, boosting typically provides better performance than bagging.
- **Risk of overfitting**: While it reduces bias and variance, boosting is more prone to overfitting, especially with noisy data.

# Bagging and Boosting

| Bagging | Boosting |
| --- | --- |
| Trains multiple models independently on different random subsets of data. | Trains models sequentially, with each model correcting errors of the previous one. |
| Reduces variance and avoids overfitting. | Reduces bias and improves accuracy. |
| Models are trained in parallel. | Models are trained sequentially (one after another). |
| Each model has equal weight in final prediction. | More weight is given to misclassified samples in subsequent models. |
| EX:Random Forest | EX:AdaBoost, Gradient Boosting |

**both bagging and boosting use N learners + yields more stable models**

# Class - 08

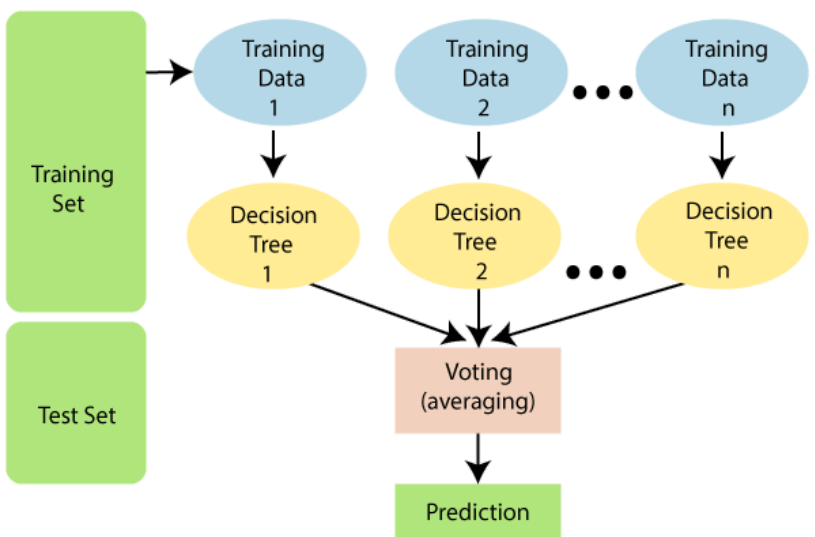
# Random Forest

# Random Forest

Random Forest is an ensemble learning algorithm used for both regression and classification tasks.

It combines multiple decision trees trained on random data subsets, using bagging (bootstrap aggregating) to improve accuracy, reduce overfitting, and enhance stability.

It is needed because individual decision trees suffer from overfitting and high variance, making them unstable.



# Random Forest

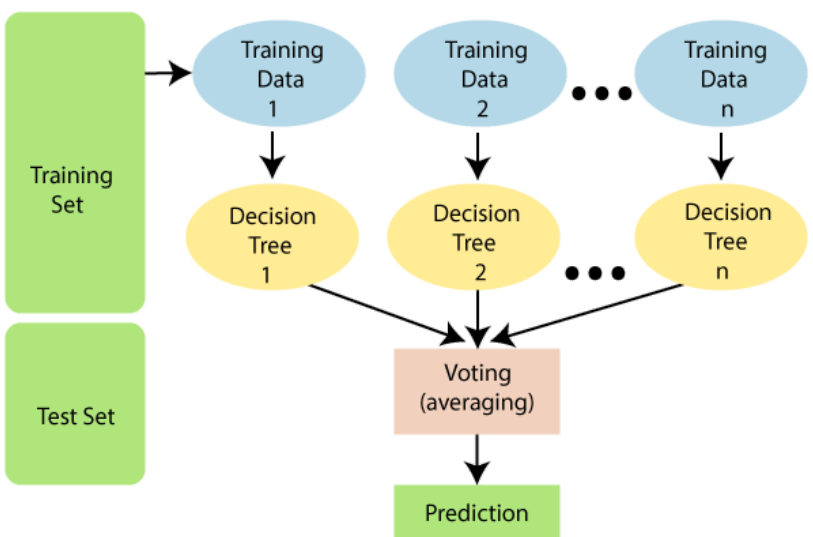
By combining multiple decision trees through **bagging**, Random Forest :

- **Reduces Overfitting**: Averages multiple trees, preventing reliance on specific patterns in training data.
- **Improves Accuracy** : Aggregating predictions leads to better generalization.

# Random Forest Regressor

A **Random Forest Regressor** is a type of Random Forest algorithm used for regression tasks. It builds multiple decision trees on random subsets of the data and averages their predictions to provide a more accurate and stable output.

- Each tree is trained on a random subset of the data using **bootstrap sampling**.
- For **regression**, the final prediction is the **average** of the predictions from all the individual trees.

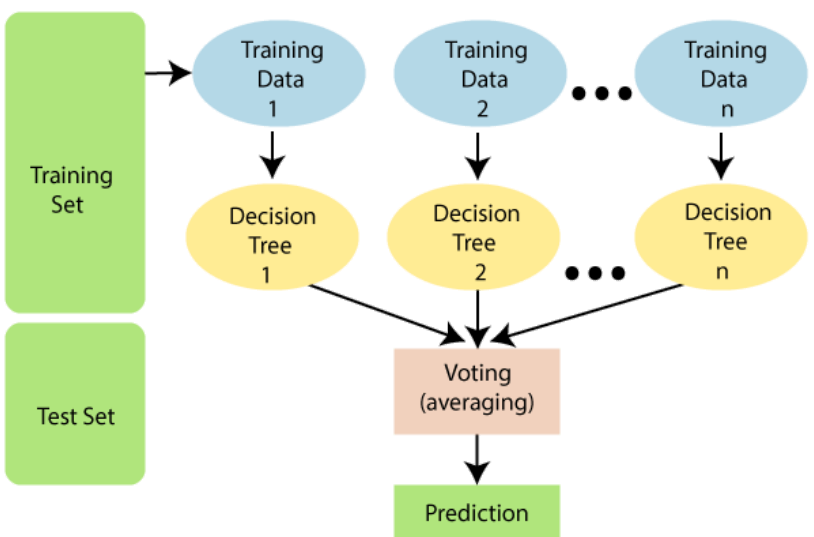


# Random Forest Classifier

A **Random Forest Classifier** is a type of Random Forest algorithm used for classification tasks.

It builds multiple decision trees on random subsets of the data and uses **majority voting** to provide a more accurate and stable output.

Each tree is trained on a random subset of the data using bootstrap sampling.



# Class - 09

# Unsupervised Learning

# Unsupervised Learning

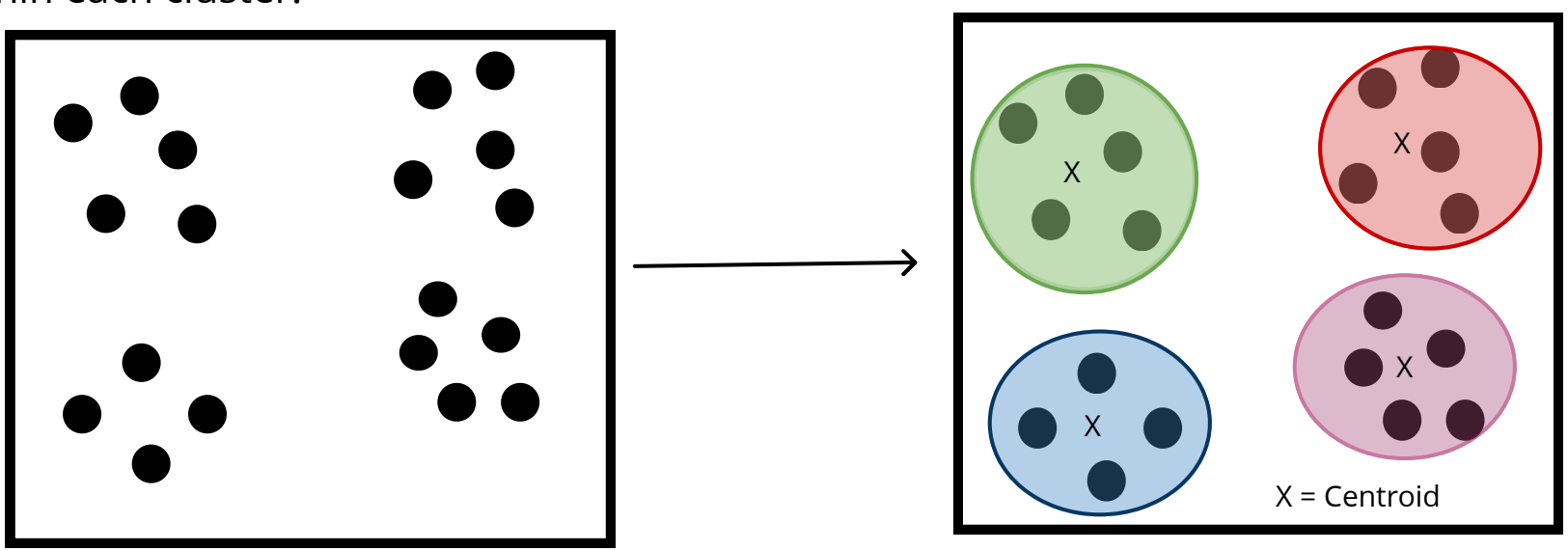
Unsupervised learning is a type of machine learning where a model is trained on **unlabeled data** (i.e., data without predefined categories or labels).

The algorithm identifies patterns, structures, or relationships within the data on its own.

# K-Means

K-Means is a popular unsupervised machine learning algorithm used for clustering data into distinct groups (clusters) based on similarities.

It partitions a dataset into K clusters, where each data point belongs to the cluster whose center (centroid) is nearest. The algorithm is iterative and aims to minimize the variance within each cluster.



# K-Means

Let's now see how K-Means actually works.

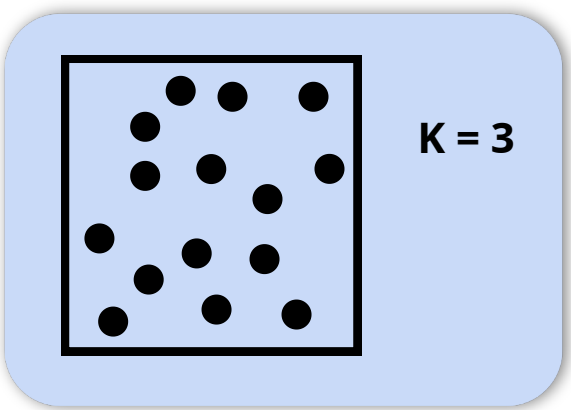
**Step1**: Choose the number of K clusters

- We decide how many groups (cluster) we want to divide our data into.
- For Ex: If we have a dataset of Customers and now we have to decide the number of clusters.

so, here we are grouping customers, now it's up to us we can

choose K = 3:

- One group for **low spenders**
- One for **medium spenders**
- One for **high spenders**



# K-Means

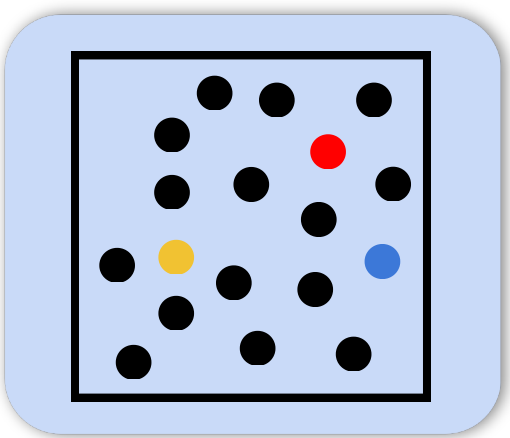
Okay, now we have selected our K value as 3.

Let's get to the next step

**Step 2**: Initialize the centroids randomly

- Now we will pick some K(K=3) random points from our dataset and these points are known as centroids.

So, here the points  are centroids which we selected randomly for our dataset.



# K-Means

Finally we have selected our centroids points

**Step 3**: Assign each data point to the nearest centroid

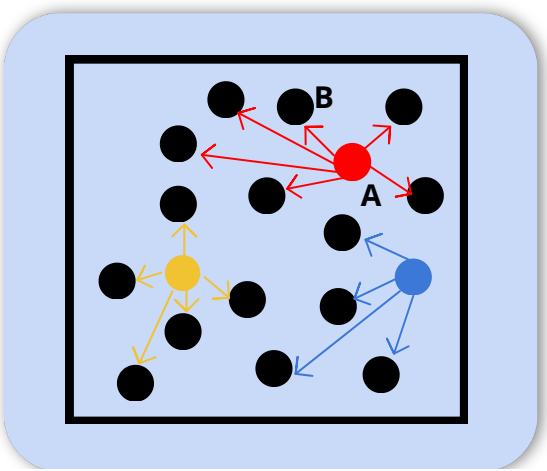
- For every point in our dataset, we will calculate the distance between the points and the centroid.
- And then we will assign that point to the nearest centroid.
- This distance is usually calculated by Euclidian Distance.

Now, the question is how we calculate this Euclidian Distance.

Euclidean Distance is just the **straight-line distance** between two points — like using a ruler

**The Euclidian Distance:** $\sqrt{(x2-x1)^2 + (y2-y1)^2}$

- $x_1$ = x-position of **Point A**
- $x_2$ = x-position of **Point B**
- $y_1$ = y-position of **Point A**
- $y_2$ = y-position of **Point B**

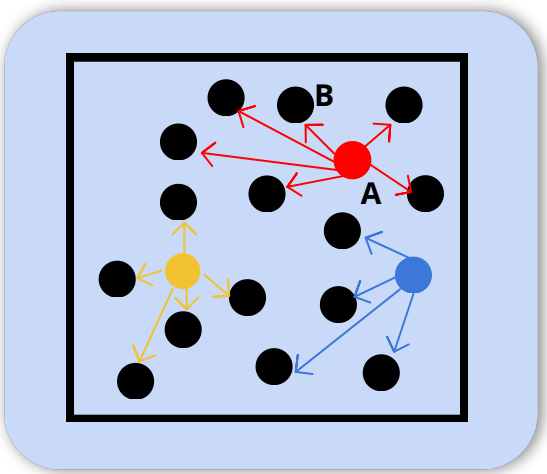


# K-Means

So, till now we have selected points closer to Centroids.
Now let's get to our next step

**Step 4**: Update the centroids

- Once we have assign datapoints to the centroids near to them.
- The new centroid is the mean (average) of all points in that cluster.



# K-Means

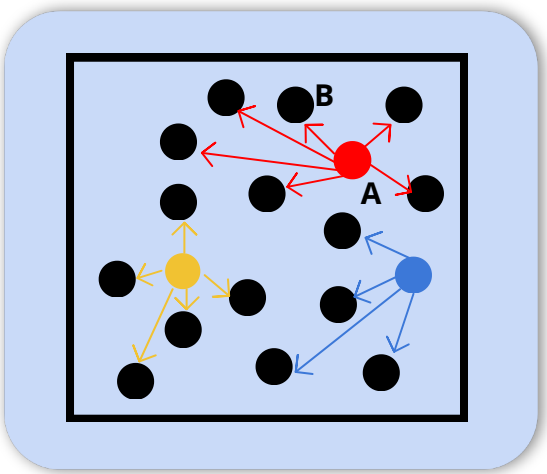
Okay, now we are about the end of the K-Means algorithim.

**Step 5**: Repeat Step 3 and Step 4

- Again we will reassign the points based on updated centroids
- Again we will Recalculate Centroids

We will keep repeating until:

- The assignment doesn't change anymore.
- Or we have reached the max. number of iterations.

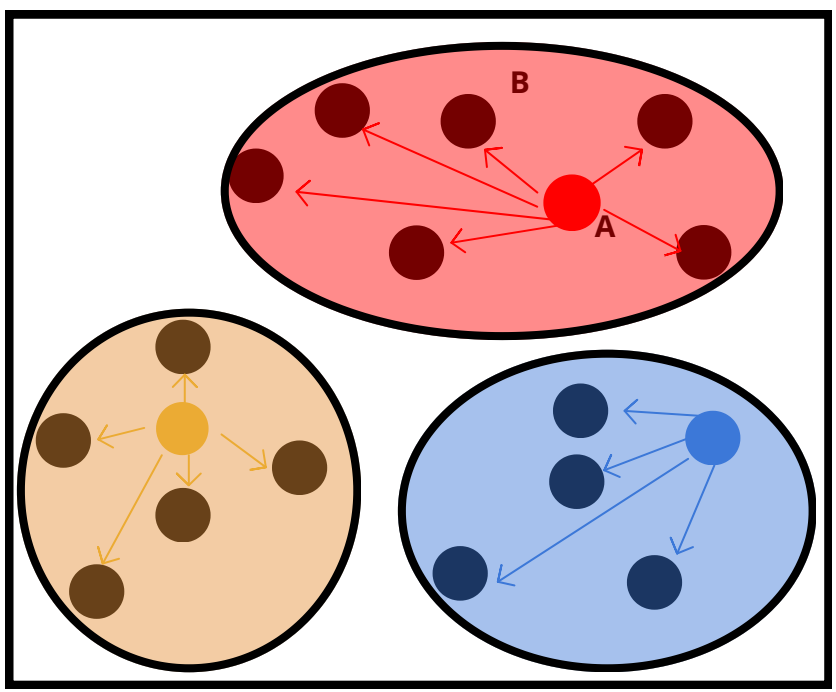


# K-Means

So, all set now. These are the final clusters we have formed.

**Step 6:** All Done

- The Algorithm stops and we have our final clusters.
- And finally we have our final K clusters



# K-Means

Now the question arises: how do we determine the number of clusters, i.e., the value of K

Now, Let's see how elbow methods help us to decide our K value.

**Step1: Run K-Means for Different Values of K**

You start by running the K-Means algorithm multiple times, each time with a different number of clusters (K).

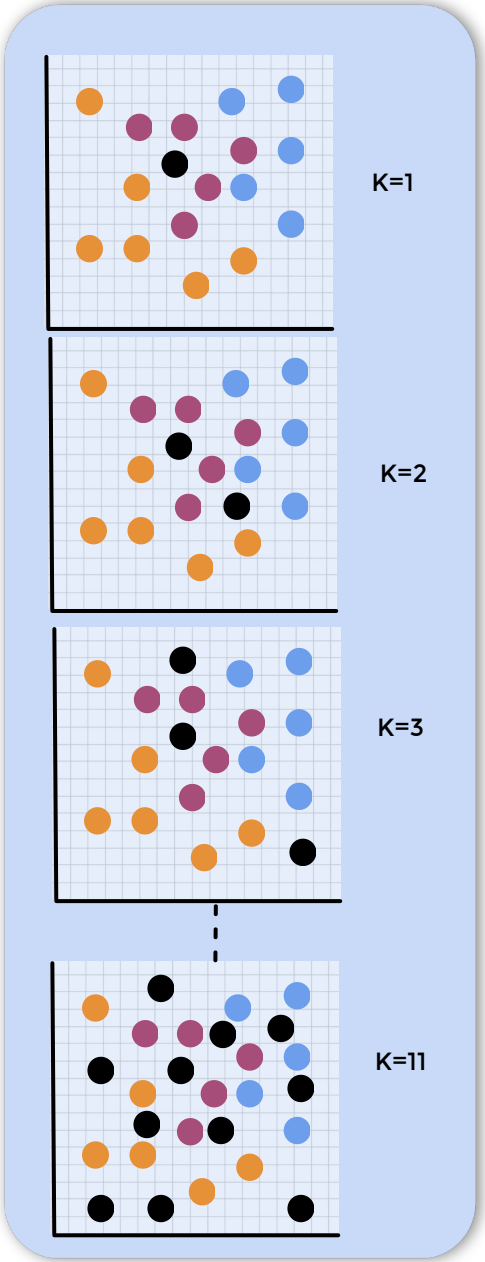
For example:

**K = 1, 2, 3, ..., 11**

The Question is why we need to choose multiple K values.

Because we don't know how many clusters our data should be divided into — so you **try multiple options** to compare.

Here, ● is our centroids.



# K-Means

**Step 2: Calculate WCSS for Each K**

After training the K-Means model for each value of K,

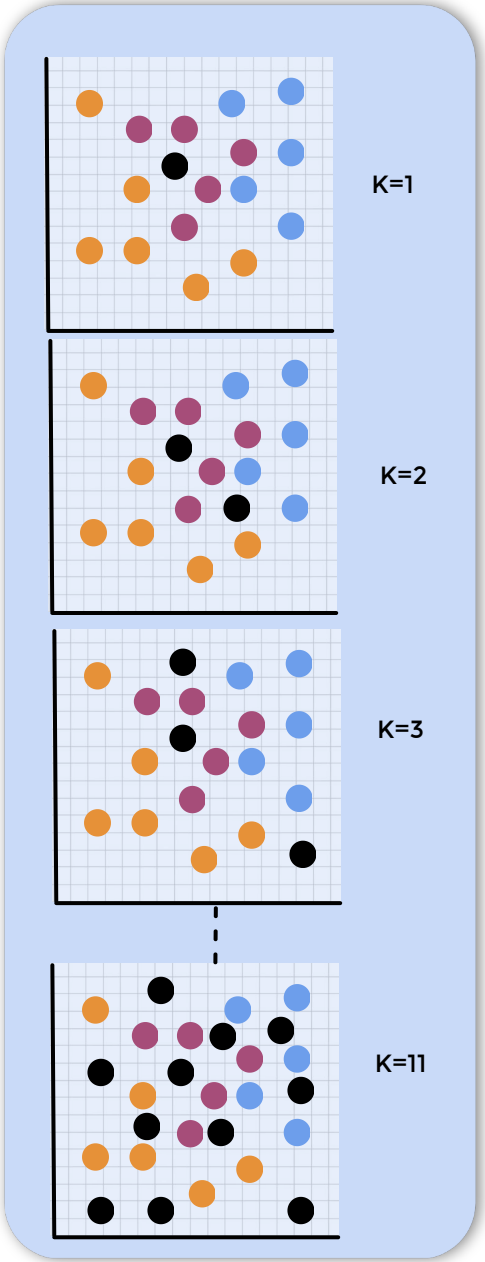
we calculate WCSS (Within-Cluster Sum of Squares) also known as inertia.

- What is WCSS?

  WCSS (Within Cluster Sum of Squares) is the total squared distance of all points from the center of their assigned cluster.

  Means how close the point is from the centroids.

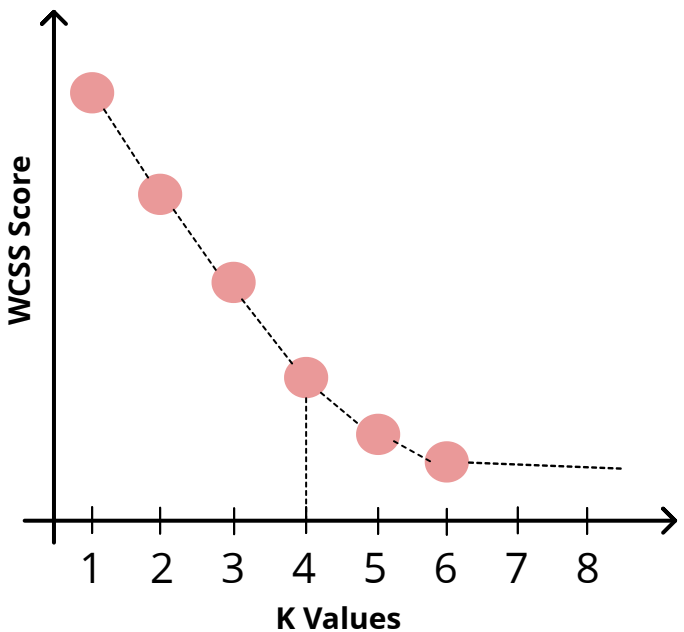
**Lower Wcss means** better clustering, but after a certain point, adding more clusters doesn't help much.



# K-Means

**Step 3: Plot WCSS vs K**

Now it's time to visualize this by plotting a graph of K(number of clusters) vs WCSS to identify the optimal number of clusters.




So, now we make a line graph where:

- X-axis = K (number of clusters)
- Y-axis = WCSS

This graph helps you visually understand how WCSS changes as K increases

Now, this graph helps us to see 2 important things:

- When K increases, WCSS decreases.
- But it doesn’t decrease at the same rate forever.

# K-Means

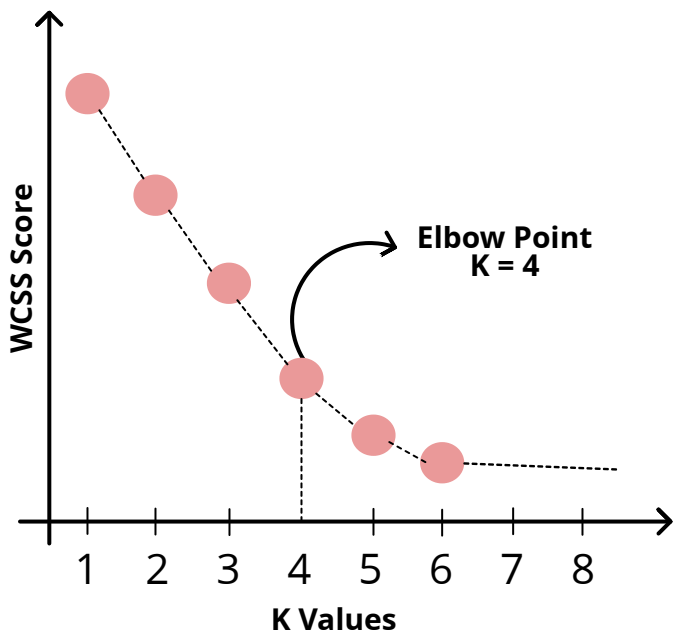
**Step 4: Find the 'Elbow Point'**

Now, look at the graph and look for a point where the graph suddenly bends like an elbow.

This is the point where:

The WCSS graph stops going down quickly.

- Adding more clusters doesn't make much difference.
- This means you've found the best number of clusters — enough to group the data properly without making it too complicated.



# K-Means

**Step 5: Choose the Optimal K**

The K value at the elbow point will be our final value for K.

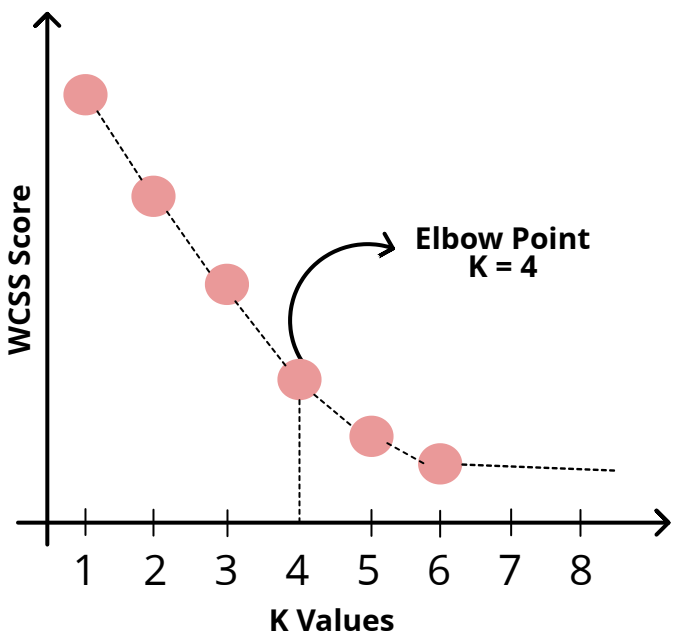
It's a balance between:

Too few clusters (high WCSS, poor separation)

Too many clusters (overfitting and complexity)

So, by seeing the graph the K value we have is 4.

**K = 4**



# K-Means

## Why Does the Elbow Method Work?

- If K is too small, clusters will have high variance, meaning data points within a cluster are far from each other.
- If K is too large, clusters will be too small and specific, leading to overfitting.
- The elbow point represents the best trade-off between variance reduction and model complexity.

# Class - 10

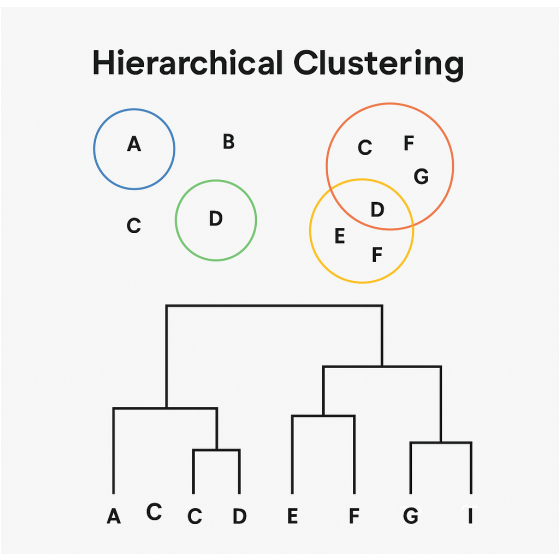
# Hierarchial Clustering

# Hierarchial Clustering

Imagine we have 5 students in our class, and we want to group students with same height.

Now, for this type of problems we are going to see Hierarchical Clustering.

It groups similar data points step-by-step and builds a tree-like structure called a dendrogram to show how and when the points were grouped.




**Type of Hierarchial Clustering:**

- **Agglomative Clustering** (Bottom-Up)
- **Divisive Clustering** (Top - Down)

# Hierarchial Clustering

**Agglomerative Clustering** is a bottom-up clustering technique.

It starts with each point as its own cluster and then keeps merging the two closest clusters step-by-step until everything becomes one big cluster.

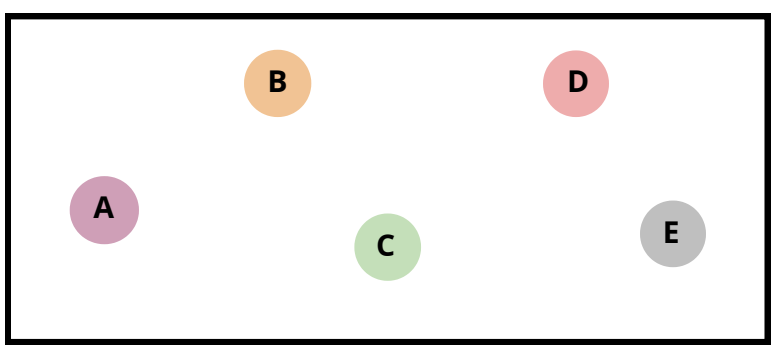
Now, let's see that example of 5 students and we have to group students with same height.

# Hierarchial Clustering

For, this let's follow some steps for Agglomative clustering:

**Step 1: Every data point is in its own group**

Every data point is its own mini cluster at the beginning.




| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

# Hierarchial Clustering

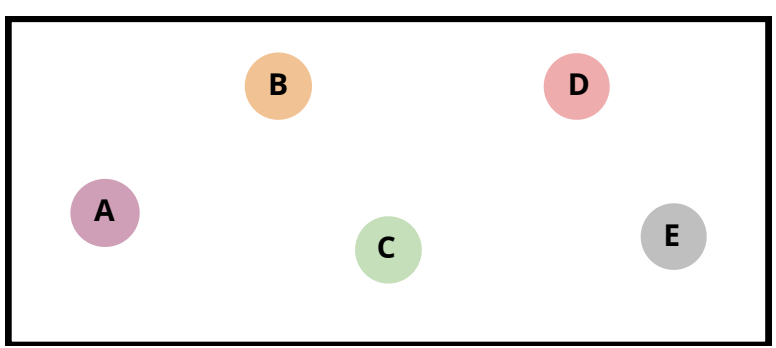
**STEP 2: Calculate Distances Between All Pairs of Clusters**

We now calculate the distance between every pair of clusters.

Since each cluster has only one point right now, We're just calculating distances between points.

Now this distance we will calculate by using Euclidian formula, but others like Manhattan distance can also be used.

**The Euclidian Distance:** $\sqrt{(x2-x1)^2 + (y2-y1)^2}$




| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

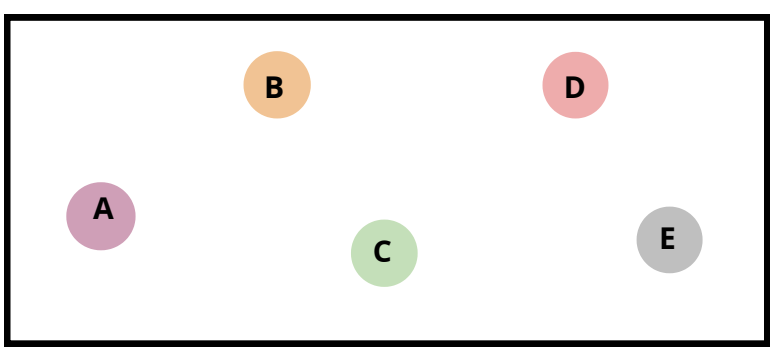
# Hierarchial Clustering

**STEP 3: Find the Closest Pair of Clusters**

Look at the distance matrix and pick the two clusters that are closest to each other.

These two clusters are the most similar → so merge them into one cluster.

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 2 | 3 | 30 | 32 |
| B | 2 | 0 | 1 | 28 | 30 |
| C | 3 | 1 | 0 | 27 | 29 |
| D | 30 | 28 | 27 | 0 | 2 |
| E | 32 | 30 | 29 | 2 | 0 |




| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

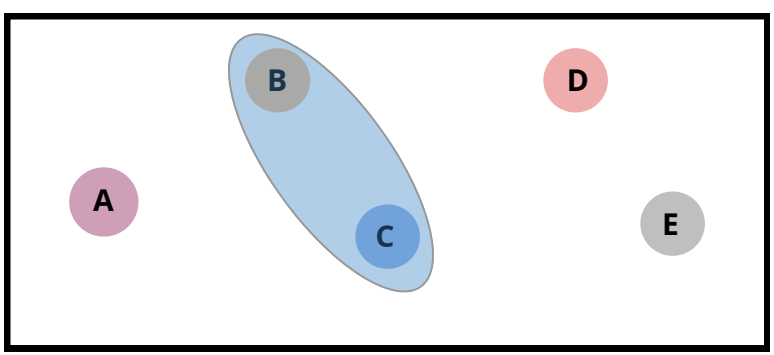
# Hierarchial Clustering

**STEP 4: Merge the Closest Clusters**

Now, in this step by seeing our distance matrix we will Combine those two closest clusters into a new, bigger cluster.

If B and C were closest (distance = 1) → merge them:

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 2 | 3 | 30 | 32 |
| B | 2 | 0 | 1 | 28 | 30 |
| C | 3 | 1 | 0 | 27 | 29 |
| D | 30 | 28 | 27 | 0 | 2 |
| E | 32 | 30 | 29 | 2 | 0 |




| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

# Hierarchial Clustering

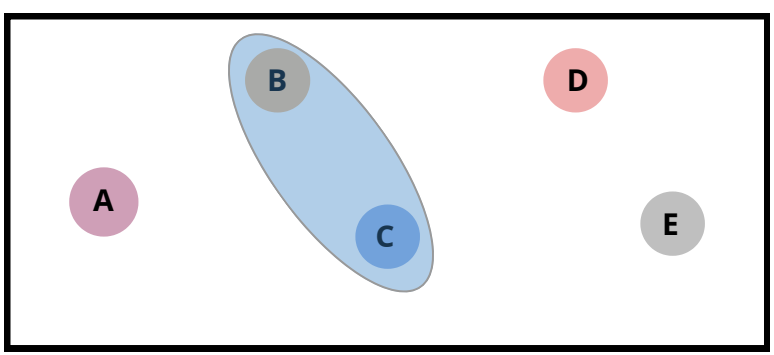
**STEP 5: Update the Distance Matrix**

So, as the data points B and C are closer to each other we merge them and call them as a cluster. Then now we will again calculate the distance for each point and then we will again make a cluster.

But how?

We use a **linkage method** to decide the new distances.

let's see about this linkage method in next slide.




| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

# Hierarchial Clustering

## Linkage Method

When we merge clusters like (B, C), how do we calculate the distance between (BC) and other clusters?

We choose a linkage method:

| | | |
|---|---|---|
| **Single** | Closest point in each cluster (min distance) | Nearest point |
| **Complete** | Farthest point in each cluster (max distance) | Worst case |
| **Average** | Average of all pairwise distances | Balanced |
| **Ward** | Minimizes total within-cluster variance | (Default in sklearn) |

# Hierarchial Clustering

**STEP 6: Repeat Steps 3–5 Until All Points Are Merged**

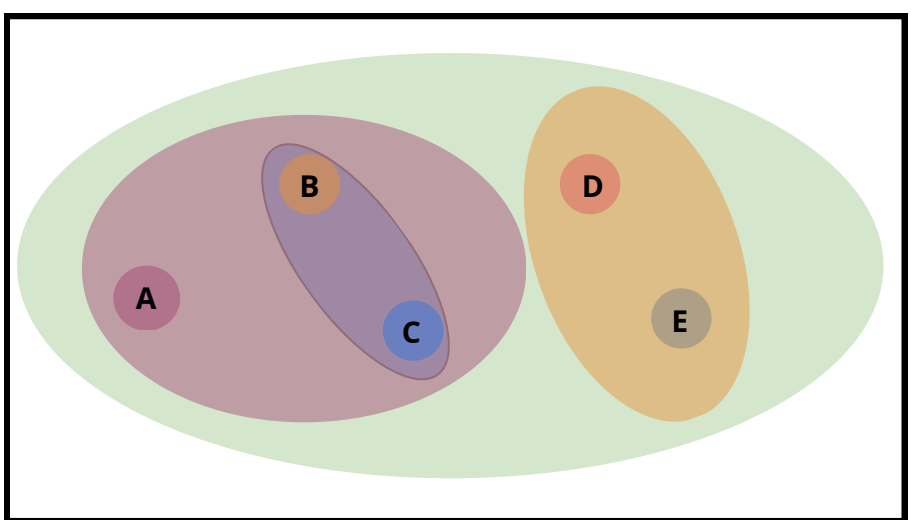
Keep finding the closest clusters, merging them, and updating distances, until:

All points are merged into one big cluster

OR

You reach the desired number of clusters (k)

So, like this we will finally be having our final clusters



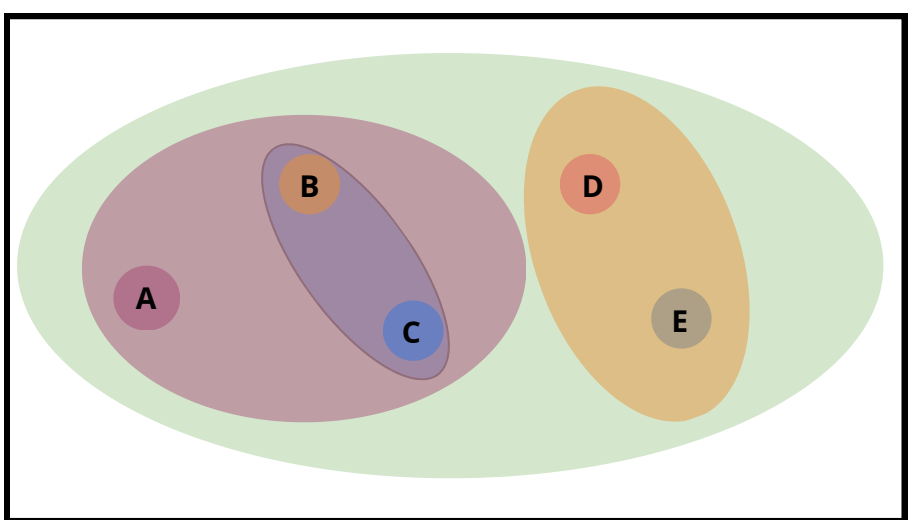

| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

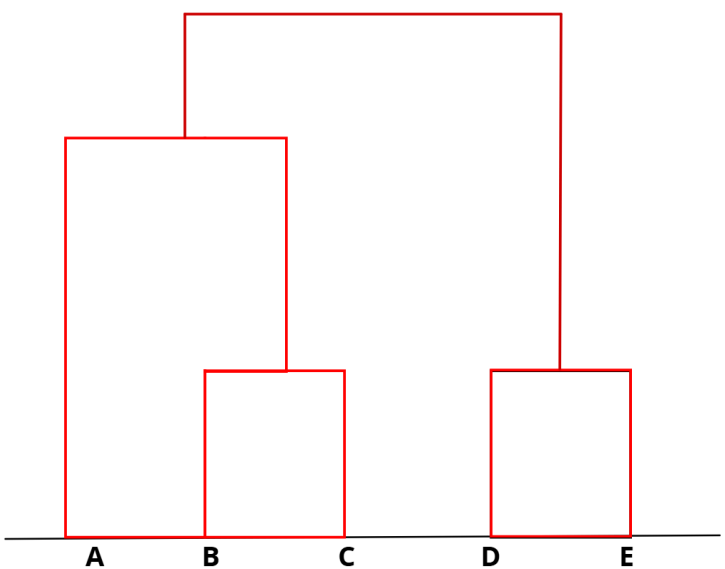
# Hierarchial Clustering

**STEP 7: (Optional) Draw a Dendrogram**

A dendrogram is a tree diagram that shows the order of merging.

- The vertical height tells we the distance at which clusters were merged.
- We can cut the tree at any level to get different numbers of clusters.a



# Hierarchial Clustering

Divisive Clustering is a top-down approach to clustering.

This method starts with all data points in one big cluster. Then it splits the cluster step by step until each data point has it's own cluster.

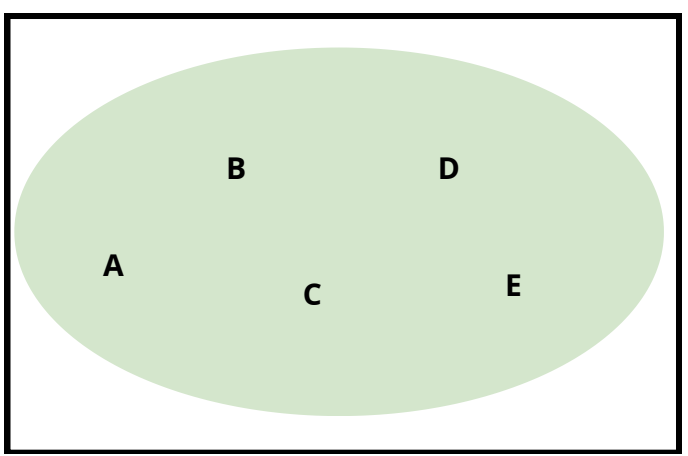
In simple terms this is a reverse version of agglomerative clustering.

Let's see it:

Again, we are having our same dataset which of height data of 5 students.

**Step 1: Start with All Points in ONE Cluster**

We first form a big cluster by merging all the data points in one cluster.



# Hierarchial Clustering

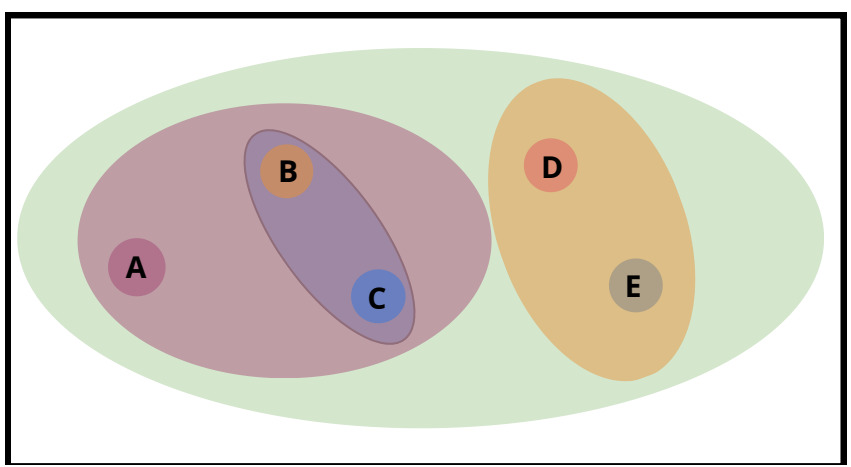
## Step 2: Choose a Way to Split It

We need to split the cluster into two most different subgroups.

How? Use a distance-based method, or even K-Means with k=2 as a sub-step.

Let's split like this:

- Group 1: [A, B, C] (low height)
- Group 2: [D, E] (high height)



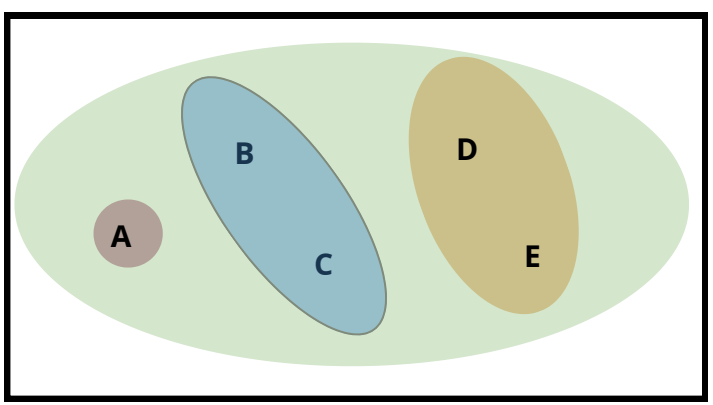

| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

# Hierarchial Clustering

## Step 3: Choose One of These Clusters to Split Again

Now pick one of the current clusters to split further.

Let's pick (ABC)




**Split again:**

A = 150

B = 152

C = 153

| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

**Split into:**

Group 1: [A]

Group 2: [B, C]

# Hierarchial Clustering

## Step 4: Keep Splitting Until Stop Condition Is Met

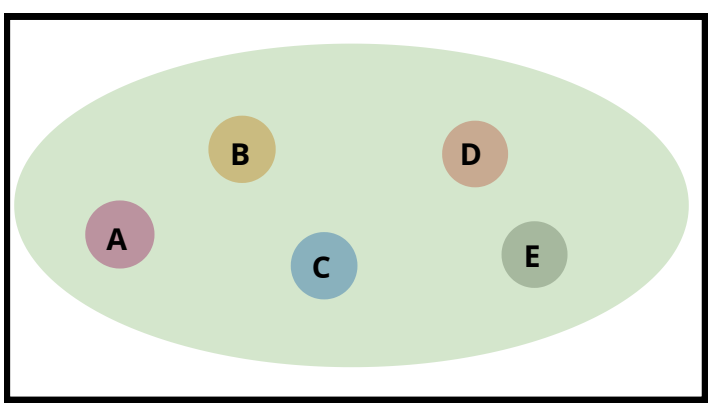
We will now split (BC) and (DE):

Group 1: [B]

Group 2: [C]

Group 3: [D]

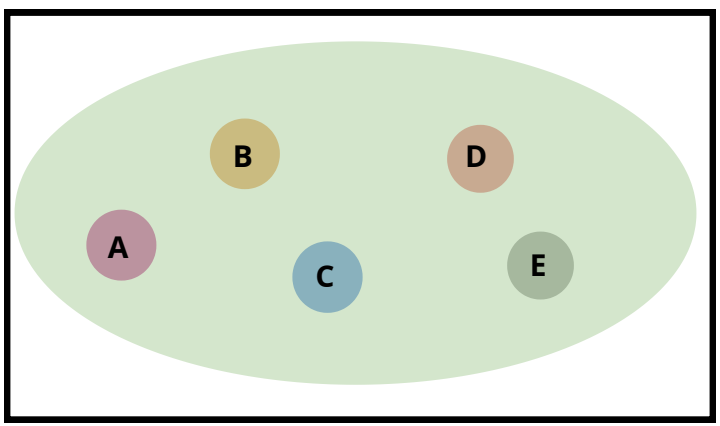
Group 4: [E]

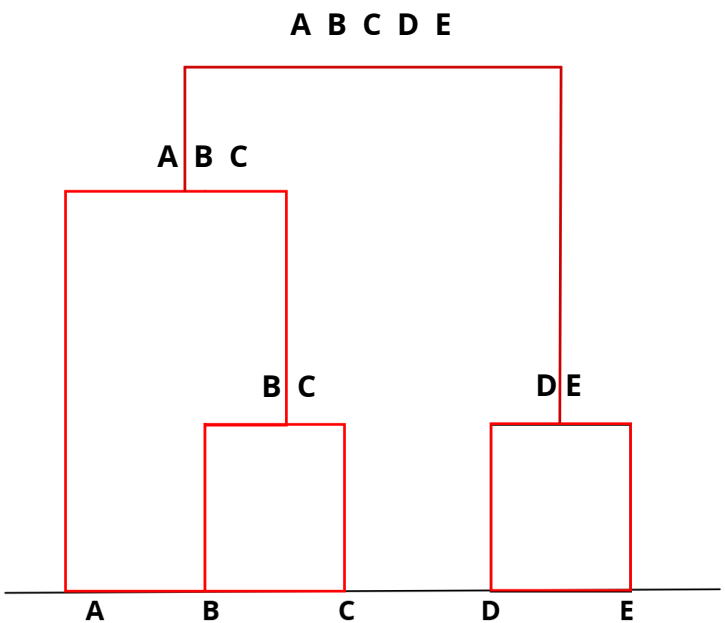



| Students | Height(in cm) |
|---|---|
| A | 150 |
| B | 152 |
| C | 153 |
| D | 180 |
| E | 182 |

# Hierarchial Clustering

Let's see how will the dendrogram of divisive clustering will look like.



# Class - 11

# DBSCAN Clustering

# DBSCAN Clustering

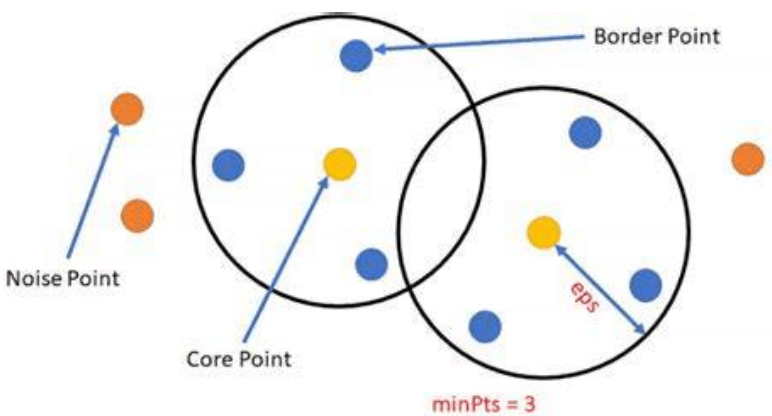
**(Density-Based Spatial Clustering of Applications with Noise)**

DBSCAN groups data points that are close to each other (dense regions) and separates outliers that don't belong to any group.

- It doesn't need you to say how many clusters you want.
- It can find arbitrary shaped clusters (not just circles like K-Means).
- It can detect noise/outliers!

What is the main goal for DBSCAN?

- Group points that are densely packed
- Mark points that are too far from others as noise



# DBSCAN Clustering

Let's get to the steps we need to follow for DBSCAN clustering.

Parameters You Must Give:

| | |
|---|---|
| eps | Radius — How far should points be to be considered neighbors |
| minPts | Minimum number of points in eps radius to form a dense cluster |

Let’s use:

- eps = 1.5
- minPts = 3

| Points | X | Y |
|---|---|---|
| A | 1 | 2 |
| B | 2 | 2 |
| C | 2 | 3 |
| D | 8 | 7 |
| E | 8 | 8 |
| F | 25 | 80 |
| G | 1 | 3 |
| H | 2 | 1 |

# DBSCAN Clustering

**STEP 1: Classify All Points**

We go point by point and count how many neighbors each has within radius = 1.5

For simplicity, just assume distance is already calculated.

Based on that, classify the point as:

| | |
|---|---|
| **Core Point** | Has at least minPts in its eps neighborhood (including itself) |
| **Border Point** | Has fewer than minPts in its eps, but is close to a Core Point |
| **Noise Point** | Not close to any Core Point |

| Points | X | Y |
|---|---|---|
| A | 1 | 2 |
| B | 2 | 2 |
| C | 2 | 3 |
| D | 8 | 7 |
| E | 8 | 8 |
| F | 25 | 80 |
| G | 1 | 3 |
| H | 2 | 1 |

# DBSCAN Clustering

**Step 2: Form Clusters**

- Start with any unvisited point
- If it's a core point, form a new cluster
- Add all the points that are density-reachable from it (connected via core points)

| Points | X | Y |
|---|---|---|
| A | 1 | 2 |
| B | 2 | 2 |
| C | 2 | 3 |
| D | 8 | 7 |
| E | 8 | 8 |
| F | 25 | 80 |
| G | 1 | 3 |
| H | 2 | 1 |

# DBSCAN Clustering

**Step 3: Expand Clusters**

- Keep visiting neighbors of the core point
- If the neighbor is also a core point, its neighbors are also added
- This way, clusters grow outward like waves

| Points | X | Y |
|---|---|---|
| A | 1 | 2 |
| B | 2 | 2 |
| C | 2 | 3 |
| D | 8 | 7 |
| E | 8 | 8 |
| F | 25 | 80 |
| G | 1 | 3 |
| H | 2 | 1 |

# DBSCAN Clustering

Step 4: Continue Until All Points Are Visited

- Repeat the above process for all unvisited points
- Mark all isolated points that don't belong to any cluster as noise

| Points | X | Y |
|---|---|---|
| A | 1 | 2 |
| B | 2 | 2 |
| C | 2 | 3 |
| D | 8 | 7 |
| E | 8 | 8 |
| F | 25 | 80 |
| G | 1 | 3 |
| H | 2 | 1 |